

# Characterization of high-resolution AI data center training workloads on single and multiple GPU nodes

Received: 23 October 2025

Accepted: 26 May 2026

Cite this article as: Elsayed, A.A., Al-Obaidi, A.A., Farag, H.E. Characterization of high-resolution AI data center training workloads on single and multiple GPU nodes. *Sci Data* (2026). <https://doi.org/10.1038/s41597-026-07496-6>

Ahmed Abd Elaziz Elsayed, Abdullah Azhar Al-Obaidi & Hany E. Z. Farag

We are providing an unedited version of this manuscript to give early access to its findings. Before final publication, the manuscript will undergo further editing. Please note there may be errors present which affect the content, and all legal disclaimers apply.

If this paper is publishing under a Transparent Peer Review model then Peer Review reports will publish with the final article.

# Characterization of high-resolution AI data center training workloads on single and multiple GPU nodes

Ahmed Abd Elaziz Elsayed<sup>1\*</sup>, Abdullah Azhar Al-Obaidi<sup>2</sup>, and Hany E.Z. Farag<sup>1</sup>

<sup>1</sup>Department of Electrical Engineering and Computer Science, York University, Toronto, Canada

<sup>2</sup>Independent Electricity System Operator (IESO), Mississauga, Canada

\***Correspondence:** elsayed7@yorku.ca

March 19, 2026

## Abstract

The rapid advancement of Artificial Intelligence (AI) is driving unprecedented computational demands, posing significant challenges to datacenter infrastructure and threatening the stability and resilience of modern power grids. This study presents an open-access dataset featuring a diverse set of AI training sessions recorded at sub-second resolution, designed to advance research on the energy consumption profiles of AI workloads and their interactions with power grid dynamics in datacenter environments. The dataset contains 32 training sessions on high-performance H100 and B200 8-GPU nodes and 40 sessions on consumer-grade NVIDIA GeForce RTX 3060 GPUs, encompassing over 1.8 million samples. Each session records power demand, CPU and GPU utilization, per-GPU power, memory usage, and temperature across diverse AI tasks, including forecasting, classification, reinforcement learning, and text and image generation. Data quality was verified through detailed technical validation, including timing accuracy, hardware limit conformance, and cross-metric correlation analysis. Measurements remained within manufacturer-specified thermal and power envelopes, and observed correlations among power, utilization, temperature, and current were consistent with

established processor and GPU behavior. The dataset provides a robust foundation for modeling AI datacenter energy behavior, system-level performance analysis, and power grid connection impact assessment studies.

## Background & Summary

The rapid expansion of Artificial Intelligence (AI) has placed unprecedented demands on datacenter infrastructure and their associated power and energy demand<sup>1</sup>. Unlike conventional enterprise data applications, modern AI training workloads are extremely resource-intensive, relying heavily on hardware accelerators such as, Central Processing Units (CPUs), Graphics Processing Units (GPUs), Tensor Processing Units (TPUs), and large-scale central processors<sup>2</sup>. These devices require extremely high power densities, which in some cases exceed 60 kW per cabinet, and when aggregated, entire AI-focused datacenters can draw tens to hundreds of megawatts of power<sup>3,4</sup>. At this scale, AI datacenters rival large industrial loads in their demand on the electrical grid, with far-reaching implications for local infrastructure design, regional energy planning, and overall system reliability.

Beyond their substantial energy consumption, AI datacenter workloads are characterized by high variability and burstiness in power demand, which create challenges for power system stability, including voltage regulation, frequency control, and overall power quality<sup>5</sup>. AI training jobs vary not only in duration but also in their computational profiles. Tasks such as image processing, natural language processing, and vertex-based regression or classification each exhibit distinct patterns of CPU, GPU, and TPU utilization, including differences in core usage, memory demands, and memory access behavior. These patterns are shaped by several factors: (1) the architecture of the AI model, such as the number and type of layers and the dimensions of input and output features; (2) the choice of hyperparameters, including optimizer type, parallelization and distribution strategies across CPUs, GPUs, and TPUs, and batch size; and (3) the architecture and capabilities of the processing units, including the number of workers, memory capacity, and communication efficiency both within and

across nodes<sup>6</sup>. Together, these factors influence the temporal power demand profile, creating fluctuations that mirror the underlying AI training workloads. These fluctuations occur across multiple timescales, from millisecond-level bursts driven by feedforward and backpropagation processes within training epochs, to minute-scale variations associated with data preparation, epoch transitions, model checkpointing, and data management. This variability complicates power and energy management at both the facility and grid levels. Capturing and analyzing the fine-grained dynamics of AI training workloads is essential for translating workload behavior into aggregate datacenter demand patterns. This process is also critical for assessing impacts on power system stability, and for informing the design and operational requirements of AI datacenters and supporting infrastructure<sup>7</sup>.

Characterizing datacenter energy consumption and power demand under AI workloads requires datasets that trace both resource utilization and power dynamics across different levels of computing infrastructure. Table 1 summarizes several publicly available datasets that have been widely used for studying workload behavior. At the cluster level, large-scale traces from Google, Azure, and Alibaba capture real-world datacenter operations. The Google Cluster dataset provides machine and task-level event logs, including job scheduling, task constraints, and measured resource usage across CPUs, memory, and storage<sup>8</sup>. Azure's traces extend this perspective by recording the full lifecycle of Virtual Machines (VM), workload metadata, and sampled utilization activity, making them valuable for studies on scheduling, capacity planning, and fault tolerance<sup>9</sup>. Alibaba's cluster dataset integrates CPU, memory, and GPU utilization with microarchitectural metrics and workload dependencies, offering a detailed view of resource allocation and system dynamics in cloud-scale environments<sup>10</sup>. While these datasets enable research on scheduling, efficiency, and large-scale workload characterization, their sampling intervals are typically coarse (minutes to hours), limiting their use for capturing the sub-second fluctuations that dominate AI training processes.

The second group includes supercomputing oriented power datasets, such as PM100 and F-DATA. PM100 covers more than 230,000 jobs from the Marconi100 supercomputer,

reporting measured power demand at the compute-node, CPU socket, and memory levels<sup>11</sup>. Similarly, F-DATA from the Fugaku supercomputer provides detailed power demand and energy consumption traces for over 24 million jobs, with per-job profiles of CPU and memory usage along with minimum, average, and maximum power values<sup>12</sup>. These datasets provide fine-grained job-level energy consumption profiles, but they primarily reflect traditional High-Performance Computing (HPC) scientific workloads rather than AI training tasks.

The third group comprises AI-specific datasets that directly capture the execution and energy consumption characteristics of AI training workloads. The DeepSeek profiling dataset provides detailed JSON-based PyTorch traces of the DeepSeek-V3 model, covering training, prefill, and decoding modes with fine-grained kernel execution and communication events<sup>13</sup>. DeepEn2023 complements this by measuring sub-second power usage on edge devices across diverse AI applications, providing kernel, model, and application level traces of workloads such as image recognition and natural language processing on mobile hardware<sup>14</sup>. However, both DeepSeek and DeepEn2023 do not include node or cluster measurements, and therefore cannot be used to characterize datacenter nodes or cluster-level behavior<sup>14</sup>. Finally, the BUTTER-E dataset focuses on deep learning across CPUs and GPUs in datacenter environments, reporting model-level energy consumption and performance trade-offs across architectures and tasks, though it lacks continuous time-series traces of training phases and sub-second high resolution profiling<sup>15</sup>.

Despite recent advances, current datasets remain insufficient for fully characterizing AI datacenter workloads. Cluster-level traces provide scale but lack the temporal resolution required to capture sub-second AI dynamics, while HPC datasets emphasize scientific workloads rather than AI training workloads. AI-specific datasets offer kernel- and model-level detail but typically omit continuous power monitoring and information on the types of workloads considered. As a result, there is limited availability of datasets that explicitly link workload type, AI model configuration, and hyperparameter settings to observed load patterns.

There remains a critical lack of high-resolution datasets that connect AI training dynamics

with datacenter-level energy consumption. This gap limits power system studies, including Connection Impact Assessment (CIA) analyses of stability, power flow, and short-circuit behavior. The absence of such data also constrains datacenter optimization, where fine-grained information is essential for energy-aware scheduling and predictive workload management. In the absence of such datasets, grid operators face uncertainty when evaluating the impact of large AI loads on system stability, while datacenter designers and AI developers lack the information needed for efficient and sustainable infrastructure scaling. Bridging this gap is therefore essential for resilient AI deployment and secure grid integration.

The dataset described in this paper was assembled to address these limitations. It focuses specifically on AI training tasks across multiple domains, including image classification, language modeling, and vertex-based learning, while systematically varying performance metrics such as batch size and model size. Workloads are executed under settings known to influence power utilization behavior, including neuron counts and optimizer selection. The dataset includes measurements from both single- and multi-GPU nodes, where multi-GPU configurations offer enhanced visibility into power dynamics observable at the facility level. In addition, the dataset provides high resolution measurements of power demand, sampled at 50 Hz (20 ms), enabling the study of sub-second variability in training workloads. Where available, the high-resolution time-series traces also include GPU utilization, memory usage, performance counters and temperature. These features make it possible to align experiments consistently and to analyze how high-resolution workload fluctuations aggregate into multi-megawatt demand profiles.

The dataset offers broad potential for reuse across multiple research and engineering domains. For power system engineers and grid operators, it provides realistic input for CIA studies, as typically performed by Independent System Operators (ISOs) and regulatory bodies such as the North American Electric Reliability Corporation (NERC). Datacenter designers and operators can use the high-resolution time-series traces to optimize workload scheduling, power provisioning, and cooling strategies under realistic AI training conditions.

| Dataset                             | Description                                                                                                                                                                           |
|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Google Cluster-Data <sup>8</sup>    | Provides large-scale traces of machine events, resource attributes, job/task events, and actual resource usage, enabling studies of cluster usage, workload behavior, and scheduling. |
| Azure Public Dataset <sup>9</sup>   | Records VM lifecycle events, placement, workload metadata, and utilization metrics, supporting research on scheduling, capacity planning, and cloud performance.                      |
| Alibaba Cluster-Data <sup>10</sup>  | Captures CPU, memory, GPU, and microarchitectural metrics, along with job arrivals, dependencies, and resource allocation for both long-running and batch jobs.                       |
| PM100 <sup>11</sup>                 | Contains power traces for 230,000 jobs on Marconi100, with node, socket, and memory level measurements for each job.                                                                  |
| F-DATA <sup>12</sup>                | Provides 24M job records from Fugaku, including CPU /memory usage and detailed job-level power and energy profiles.                                                                   |
| DeepSeek Profile Data <sup>13</sup> | Offers PyTorch profiling traces of DeepSeek-V3 training and inference, detailing compute-communication overlap at fine granularity but without datacenter-level power metrics.        |
| DeepEn2023 <sup>14</sup>            | Measures sub-second energy consumption of AI workloads on smartphones, with kernel-, model-, and application-level traces for diverse tasks.                                          |
| BUTTER-E <sup>15</sup>              | Provides power and energy usage of deep learning models across CPUs and GPUs, linking efficiency with architecture and workload design choices.                                       |

Table 1: Published AI Workload Datasets.

AI developers and performance engineers may employ the dataset to evaluate model scaling efficiency, energy-to-accuracy trade-offs, and runtime behavior under different hardware configurations. By providing empirical traces that capture the full spectrum of AI workload variability, this dataset establishes a foundation for cross-disciplinary research on datacenter efficiency, sustainable AI development, and the integration of AI facilities into modern power systems.

## Methods

The primary objective of the dataset is to capture both static and dynamic characteristics of AI training workloads, including high-resolution time-series traces of processing units

during training. Collected at both the single-machine and node-scale, the dataset accounts for variations across AI architectures, training objectives, hyperparameter choices, and processing unit capabilities. It is designed to support a broad range of research questions, spanning AI datacenter workload characterization, infrastructure planning, and CIA studies in power systems.

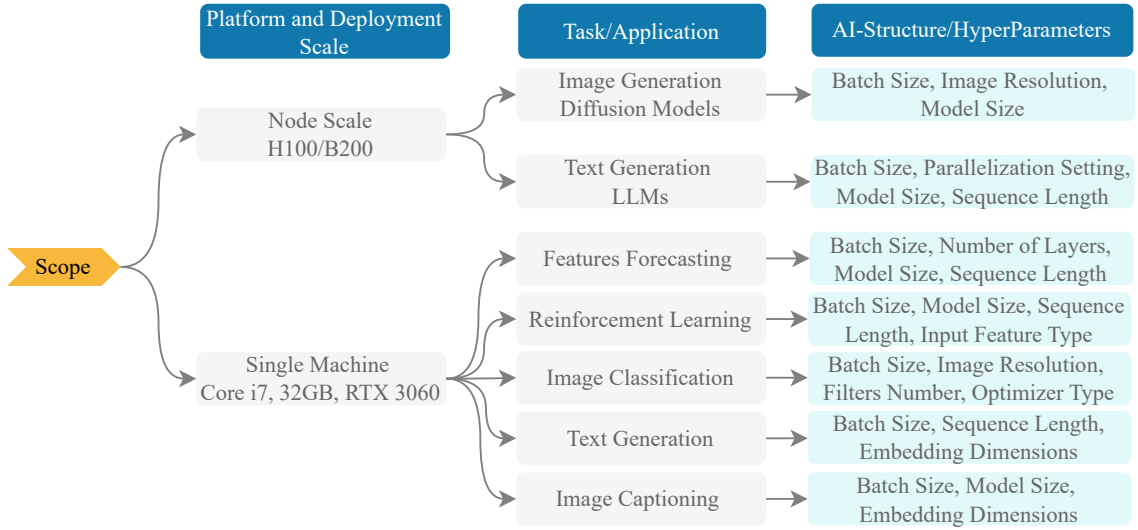


Figure 1: AI training workloads dataset scope including environment, AI applications and AI model architecture and hyperparameters

## Measurements and Dataset Scope

Figure 1 illustrates the overall scope of the AI training workload dataset, which is structured around three principal aspects: (1) platform and deployment scale, (2) applications and training objectives, and (3) AI architectures and hyperparameters. The experiments were designed based on these aspects to provide an accurate representation of AI training workloads across diverse computational environments. The platform and deployment scale aspect covers environments ranging from single-CPU and single-GPU systems to multi-GPU and multi-virtual CPU (vCPU) nodes. The applications and training objectives aspect includes a range of AI workloads such as image generation, text generation with LLMs, and feature



| Type | Local Single Machine Environment                | Datacenter Node Environment                     |                        |
|------|-------------------------------------------------|-------------------------------------------------|------------------------|
|      |                                                 | H100                                            | B200                   |
| CPU  | 12th Gen Intel(R)<br>Core(TM) i7-12700 2.10 GHz | VM-Lambda Stack 22.04<br>208 vCPU <sup>17</sup> |                        |
| GPU  | NVIDIA GeForce<br>RTX 3060 (12 GB)              | 8xH100 SXM<br>(80GB VRAM)                       | 8xB200<br>(180GB VRAM) |
| RAM  | 32 GB                                           | 1800 GB                                         | 2900 GB                |
| OS   | Windows 10, 64-bit,<br>x64-based processor      | Ubuntu Server 22.04, x86-64                     |                        |

Table 2: Specifications of tested machines.

forecasting. The workloads are assigned to appropriate environments according to their computational requirements, where tasks that demand substantial computing resources, such as image generation and LLM training, are executed in node-scale environments, while tasks requiring less computation, such as forecasting and image captioning, are conducted in single-machine environments. The AI architectures and hyperparameters aspect includes variations in batch size, model size, image resolution, sequence length, embedding dimensions, optimizer type, and number of layers or filters. These variations enable systematic evaluation of how architectural and training configurations affect performance metrics and workload dynamics. Table 2 summarizes the specifications of the environments used for each experimental configuration. For the node-scale experiments, a Lambda Cloud on-demand GPU instance Virtual Machine (VM) with the Lambda Stack 22.04 image was used<sup>17</sup>. The VM provides 208 virtual CPU cores (vCPUs) and 8 GPUs (H100 or B200), running on Ubuntu Server 22.04 with an Intel processor based on the x86-64 architecture. The Lambda Stack 22.04 image includes pre-installed versions of Python, the NVIDIA driver, CUDA, and PyTorch. Detailed information about the system image and configuration is available at <https://docs.lambda.ai/public-cloud/on-demand/>. In addition, for the node-scale experiments, an environment.yml file is provided, which lists the complete software environment, packages, and dependencies required for each task. This setup is described in the AI training and monitoring tools setup section to support reproducibility of the experiments.

High-resolution time-series traces are collected to enable accurate AI datacenter workload

characterization and pattern identification across diverse applications, model architectures, and hyperparameter settings. The dataset is constructed by monitoring CPU and GPU performance metrics during AI training workloads, with the following key measurements:

- *GPU Utilization (%)*: The percentage of time within the sampling interval at which any streaming multiprocessor on the GPU is actively executing instructions, as opposed to being idle or stalled.
- *CPU Utilization (%)*: The percentage of the total available processing capacity across all CPU cores actively engaged in computation.
- *GPU Memory Utilization (%)*: The percentage of the GPU's dedicated VRAM currently allocated, indicating memory pressure regardless of the actual content stored.
- *GPU Memory Used (MB)*: The absolute amount of GPU memory allocated by active processes, representing the actual volume of data residing in VRAM.
- *GPU and CPU Power Demand (W)*: The instantaneous electrical power drawn by the processing device (GPU or CPU), measured in watts (W). It represents the rate of energy consumption during operation and is directly correlated with both computational activity and heat generation within the device.
- *GPU and CPU Temperature (°C)*: The operating temperature of the device's core or junction, reported in degrees Celsius.
- *Virtual and Physical Memory Commitment*: The ratio of committed virtual address space to available physical RAM, indicating the pressure on memory resources.
- *CPU Core-Level Analysis*: Detailed examination of workload distribution across individual CPU cores, identifying utilization patterns, load-balancing efficiency, and potential single-core bottlenecks.

A detailed list of all recorded performance metrics and configuration parameters is provided in the *Data Records* section

## AI Training and Monitoring Tools Setup

The data generation setup comprises two main phases. The first focuses on the AI training workloads, encompassing diverse training codes and environments across various applications, with the flexibility to modify AI architectures and hyperparameters. The second phase involves the monitoring tools, which record detailed CPU and GPU performance metrics and runtime signals during training. To minimize measurement perturbation, where aggressive instrumentation can distort the execution characteristics of AI training workloads, the monitoring phase is decoupled from the training phase. Consequently, kernel-level profilers such as the PyTorch Profiler are not used, as their intrusive nature and high overhead can bias workload behavior and invalidate performance measurements.<sup>16</sup>

### Training Workloads

For the AI training workloads, open-source and well-documented examples are employed to ensure accessibility, reproducibility, and transparency of both the training codes and environments. The training codes are available through public repositories<sup>18–22</sup>, and are also provided alongside the training datasets in the main data repository<sup>25</sup>, as detailed in the *Data Records* section. In the single machine experiments, MATLAB 2024b is used to train AI models for forecasting, reinforcement learning, image classification<sup>18</sup>, text generation<sup>19</sup>, and image captioning<sup>20</sup>. At the node-scale, resource-intensive tasks including, high-resolution image generation using diffusion models<sup>21</sup>, and LLM text generation<sup>22</sup> are included, as these workloads are cluster-dominant and demand significant computational resources. For multi-CPU and multi-GPU training environments, Python is used with PyTorch Lightning and LLaMA Factory to manage distributed training efficiently. Table 3 provides an overview of the AI training workloads and applications included in the dataset, while Table 4 outlines the

model architectures and hyperparameters considered. Each model and its hyperparameters are varied across multiple configurations, representing low, moderate, and high computational loads. The definitions of these parameters, along with their significance in studying AI training workloads, are summarized below:

| Task                              | Programming Language     | Training Workload Description                                                                                |
|-----------------------------------|--------------------------|--------------------------------------------------------------------------------------------------------------|
| Single Machine Scale              |                          |                                                                                                              |
| Feature Forecasting               | MATLAB 2024b             | AI regression model for solar farm system energy generation forecasting using weather data.                  |
| Reinforcement Learning            |                          | AI reinforcement learning agent trained to control power system frequency during sharp wind power variation. |
| Image Classification              |                          | AI classification model using convolutional neural network for MNIST numbering handwriting <sup>18</sup> .   |
| Text Generation                   |                          | AI text generation model trained using words embedding <sup>19</sup> .                                       |
| Image Captioning                  |                          | AI image captioning model combining both image and text AI training workload <sup>20</sup> .                 |
| H100/B200 Node Scale              |                          |                                                                                                              |
| Image Generation Diffusion Models | Python/PyTorch Lightning | Generative AI model for high resolution image generation on large scale dataset <sup>21</sup> .              |
| Text Generation LLMs              | Python/LLaMA Factory     | LLMs text generation models and reasoning on large scale dataset <sup>22</sup> .                             |

Table 3: AI training workloads

- *Batch Size*: The number of training samples processed before updating model parameters. This choice directly affects convergence speed, training stability, memory utilization, and power demand, making it a key factor in shaping AI workload patterns.
- *Model Size (Number of Learnable Parameters)*: The total count of learnable parameters (weights and biases) in an AI model. Model size governs computational demand, memory requirements, training time, and energy consumption, while also influencing achievable performance.
- *Input Sequence Length*: The number of tokens or time steps processed per input. Longer sequences improve contextual understanding but quadratically increase computational

| Task                                 | AI Architecture and Hyperparameters value                                                                                      |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Single Machine Scale                 |                                                                                                                                |
| Feature Forecasting                  | Batch size: (50,100,150), Model Size: (474K,1.6M,3.6M)<br>Input Sequence Length: (96,192,672), # Layers: (6,8,10)              |
| Reinforcement Learning               | Batch size: (150,250,350), Layer Size: (256,512,1024)<br>Input Sequence Length: (150,250,350), Input Type: (Feature, Sequence) |
| Image Classification                 | Batch size: (750,1500,2250), Image Size: (112,224,280)<br># Filters: (8,16,32), Optimizer Type: (Adam, SGD, RMS)               |
| Text-Generation                      | Batch size: (32,128,512), Input Sequence Length: (100,250,500)<br>Embedding Dimension: (100,300,1000)                          |
| Image Captioning                     | Batch size: (128,256,1024), Layer Size: (512,1024,2048)<br>Embedding Dimension: (256,512,1024)                                 |
| H100/B200 Node Scale                 |                                                                                                                                |
| Image Generation<br>Diffusion Models | Batch size: (128,256,512), Image Size: (32, 64, 128)<br>Model Size: (107M, 470M, 1.7B)                                         |
| Text-Generation<br>LLMs              | Batch size: (2,16,32), Parallelization Settings:ds(Z1, Z2, Z3)<br>Cutoff length: (1024,2048,4096), Model Size: (1B, 3B, 8B)    |

Table 4: AI architecture and hyperparameters

and memory demands in attention-based architectures, affecting GPU utilization and AI workload patterns.

- *Layer Width (Filters or Embedding Dimension)*: The number of neurons per layer, expressed as filters in convolutional layers or embedding dimensions in token-based models. Increasing width enhances representational capacity but proportionally raises memory and computational costs.
- *Number of Layers (Network Depth)*: The depth of a neural network, defined by the number of stacked layers. Deeper networks capture more complex features but substantially increase training time, computational cost, and power usage.
- *Image Size*: The spatial resolution of input images. Higher resolutions improve detail and accuracy in vision tasks but lead to quadratic growth in computation and memory usage, heavily impacting GPU load and training workload.
- *Optimizer Type*: The algorithm used to update model parameters by minimizing the

loss function. Optimizer choice shapes convergence behavior, memory overhead, and energy consumption.

- *Parallelization Settings:* The configuration defines how training is distributed across multiple GPUs within a node. The notations ds(Z1), ds(Z2), and ds(Z3) refer to the DeepSpeed distributed training configurations used in the experiments. Specifically, Z1 corresponds to the Zero Redundancy Optimizer (ZeRO) Stage-1, Z2 to ZeRO Stage-2, and Z3 to ZeRO Stage-3 optimization<sup>23</sup>. The ZeRO framework reduces memory redundancy and improves utilization of the aggregate GPU memory within a node by partitioning optimizer states, gradients, and model parameters across GPUs. In conventional training, each GPU maintains a full copy of the model and its associated states, whereas DeepSpeed ZeRO distributes these components across GPUs according to the selected stage to enable more memory-efficient training. The configuration settings used in this study are available in the LLaMA-Factory GitHub repository<sup>22</sup>.

## Monitoring Tools

To ensure accurate monitoring of CPU and GPU resources during AI training, the monitoring tools were selected based on two primary considerations: (1) avoiding interference with the training code by using a tool that communicates directly with hardware sensors through the System Management Bus (SMBus) or Inter-Integrated Circuit (I<sup>2</sup>C) interfaces of CPU and GPU components, and (2) employing separate lightweight monitoring environments to minimize measurement overhead<sup>16</sup>. Accordingly, two complementary monitoring approaches were adopted for workload measurement.

1. *HWiNFO-based Monitoring:* HWiNFO is a Windows-based monitoring tool that uses a signed kernel-mode driver to directly interface with hardware components. It collects measurements from CPU Model-Specific Registers (MSRs), memory-mapped registers, and embedded controller chips through SMBus/I<sup>2</sup>C communication<sup>26</sup>. This approach

is particularly suited to single-machine setups, where it provides high-resolution, low-latency monitoring. By operating at kernel mode, HWiNFO enables accurate monitoring, including CPU core and package temperatures, core frequencies, C-states and P-states, package power, and memory controller load. It also monitors motherboard voltages, fan speeds, GPU utilization (compute and memory), GPU core temperature and fan speeds, board power draw and limits, GPU/Memory/SM clocks, voltages, ECC errors, and active process information. Measurements can be captured at reporting intervals as low as 100 ms.

2. *Python Package-based Monitoring:* To support lightweight and cross-platform monitoring (mainly Linux based node-scale), a Python-based environment was developed. This method relies on several packages: **psutil** for CPU utilization and frequency tracking, **os** for CPU energy and power statistics, and **pynvml** for accessing NVIDIA driver and firmware data. Through these tools, GPU metrics such as utilization, memory usage, power demand (both percentage and absolute values), and temperature can be collected. The achievable reporting rate is up to 20 ms, constrained by the internal update frequency of the NVIDIA driver. The custom Python monitoring code in the GitHub repository<sup>25</sup> (i.e., path "training code\ custom\_logging.py") illustrates the implementation for capturing CPU and GPU parameters. This Python-based approach provides scalable, low-overhead monitoring that minimizes interference with AI training workloads.

For node-scale training, an H100 and B200 8-GPU node was employed using a Lambda AI online instance<sup>27</sup>. This platform was selected for its browser-based control interface, which eliminates the need for SSH command-line management and instead provides a Jupyter Notebook environment for Python code execution. This setup facilitates streamlined, trackable training, as well as convenient uploading and downloading of recorded AI workload sessions, without requiring complex system configurations.

## Data Records

The dataset, available at AI workload profile<sup>24</sup>, consists of three main folders: Data\_Visualization\_&Parameters\_Description, Single\_Machine\_Dataset, and Node\_Dataset. This section provides a detailed description of the content and organization of these folders.

The folder Data\_Visualization\_&Parameters\_Description includes three files 1) Single\_Machine\_Analysis.mlx, which is a MATLAB Live Editor Code that provides the data visualization and statistical analysis for the "Single\_Machine\_Dataset" folder; 2) Node\_Analysis.mlx, which provides the data visualization and statistical analysis codes for the "Node\_Dataset" folder; and 3) Parameters\_Description.xlsx, which provides a detailed list of the measured metrics, units, and description using HWiNFO tool and the Python Monitoring Code. A summary of the available metrics is provided below:

- HWiNFO: Date and Time (in YYYY-MM-DD, HH:MM:SS.mS format), Virtual Memory Committed [MB], Virtual Memory Available [MB], Virtual Memory Load [%], Physical Memory Used [MB], Physical Memory Available [MB], Physical Memory Load [%], and Page File Usage [%]. CPU metrics include Core VIDs [V] for performance cores (P-core 0-7) and efficiency cores (E-core 8-11), Core Usage [%] and Utility [%] per thread, Core Ratios [x], Core Temperatures [°C], Distance to TjMAX [°C], Thermal Throttling [Yes/No], Critical Temperature [Yes/No], and Power Limit Exceeded [Yes/No] statuses. Power measurements include CPU Package Power [W], IA Cores Power [W], GT Cores Power [W], System Agent Power [W], and various power demand limits. GPU metrics cover GPU Temperature [°C], GPU Power [W], GPU Core Load [%], Memory Controller Load [%], GPU Memory Usage [%], various utilization metrics (D3D, Video Decode/Encode, Computing), Performance Limiters [Yes/No], Fan Speeds [RPM/%], and memory allocation statistics. Additional monitoring includes PCIe Link Speed [GT/s], Frame Rates [FPS], Network Usage [KB/s], Drive Health [%], and various voltage/current readings across system components.



- Python Code Metrics: timestamp (in YYYY-MM-DD HH:MM:SS.mS format), CPU Utilization (%), and CPU Frequency (MHz). For each GPU, the following performance metrics are tracked: GPU Utilization (%), GPU Memory Utilization (%), GPU Power TDP (%), GPU Memory Used (MB), GPU Memory Total (MB), GPU Power Demand (W), and GPU Temperature (°C).

The "Single\_Machine\_Dataset" contains raw data from 40 AI training workload monitoring sessions, organized across five application domains: Feature Forecasting, Reinforcement Learning, Image Classification, Text Generation, and Image Captioning. Each application is stored in a dedicated folder, which further contains subfolders corresponding to different AI model architectures and hyperparameter settings, as summarized in Table 4. Every session is recorded as a CSV file, providing high-resolution time-series traces collected through the HWiNFO tool. In addition, the MATLAB training code used for the single-machine experiments is included in a separate folder named "Training Code", ensuring reproducibility of the results.

The "Node\_Dataset" comprises raw data from 32 AI training workload monitoring sessions, focusing on two large-scale applications that dominate cluster-level training: Image Generation using Diffusion Models and Text Generation using LLMs. Each application is organized in a dedicated folder, which contains three subfolders: "H100", "B200", and "Training Code". The "Training Code" folder includes Linux terminal commands, environment configuration files, dependency specifications, Python training scripts, and the Python-based monitoring tool. The "H100" and "B200" folders are further divided into subfolders corresponding to different AI model architectures and hyperparameter settings, consistent with those summarized in Table 4. Each monitoring session is stored as a CSV file containing high-resolution time-series traces collected using the Python monitoring code "custom\_logging.py". It is important to note that CPU power and CPU temperature measurements could not be captured in the node-scale experiments because the virtualized server environment did not provide access to the required CPU sensors. As a result, these fields are retained in the monitoring output

format, but the corresponding columns remain empty in the released node-scale CSV files. The Python monitoring code in the GitHub repository<sup>25</sup> keeps these fields so that researchers with full hardware access can record them when running the same code in non-virtualized environments.

## Technical Validation

A rigorous set of validation procedures was carried out to ensure the accuracy, precision, and consistency of the recorded measurements during AI training workloads. These procedures examined multiple aspects of the dataset, including sampling stability, operational validity, measurement consistency, physical realism, and performance scaling. First, the timing fidelity of the monitoring tools was evaluated by analyzing the distribution of sampling intervals, confirming that sub-second resolution was consistently maintained across all recording sessions. Next, hardware power and thermal readings were assessed against manufacturer-defined limits to verify that all values remained within safe and non-throttling operating ranges. Power measurements reported by separate measurement sources (HWiNFO and Python-based tool) were cross-compared for both CPU and GPU platforms to validate consistency across tools. To assess physical realism, statistical correlation analyses were performed between power, utilization, temperature, and current across varying workloads and configurations. Finally, the effects of model and workload configurations were examined to ensure predictable scaling behavior, confirming that the dataset captures realistic performance dynamics under different computational loads. These analyses collectively demonstrate that the dataset accurately reflects expected component behavior and interdependencies, providing a reliable foundation for future modeling and analysis.

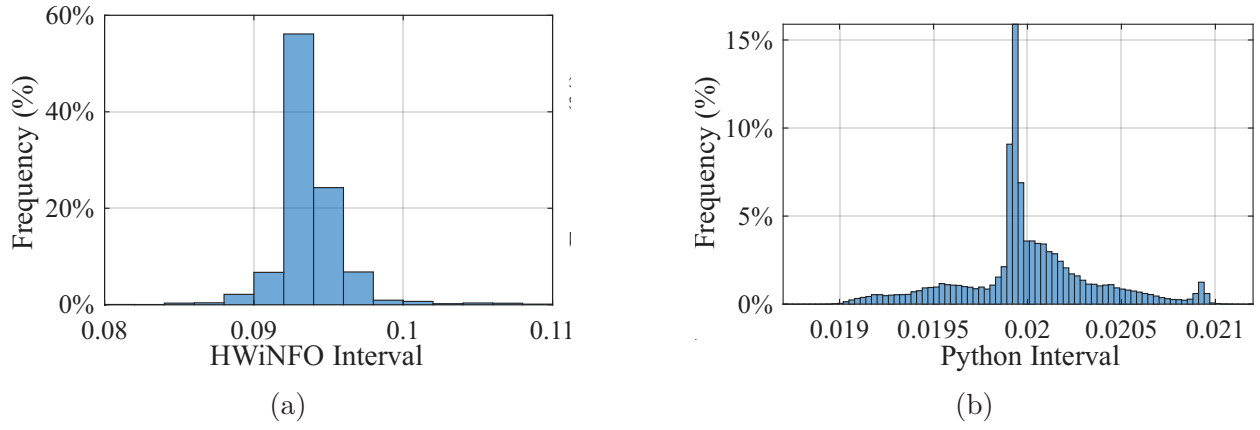


Figure 2: Distribution of reporting intervals for (a) HWiNFO, and (b) Python monitoring tools.

## Sampling Stability

Across all monitored performance metrics, no missing or corrupted samples were detected, confirming 100% data integrity for the reporting duration. Furthermore, the temporal accuracy of the dataset was verified through reporting-interval and consistency tests. The stability of the sampling process was assessed by computing the time difference between consecutive data samples. Both HWiNFO and the Python monitoring code record timestamps with sufficient precision for accurate sample alignment. The Python time package supports sub-millisecond timestamp precision, while HWiNFO provides millisecond-level timing adequate for reliable temporal labeling of sensor data. Figure 2 presents the distribution of reporting intervals for both tools. HWiNFO samples at a nominal 100 ms, with most intervals 90–95 ms, indicating stable reporting. The distribution is narrow, indicating that over 90 % of the recorded intervals remain close to the nominal rate, confirming stable reporting behavior. In contrast, the Python-based monitoring script uses a 20 millisecond interval (50 Hz) and maintains a tighter distribution centered around 20 milliseconds, with most samples within  $\pm 1$  millisecond of the target value. This demonstrates high temporal precision and minimal variation in sampling intervals. The difference in sampling stability is primarily attributed to implementation details. HWiNFO collects a large number of performance metrics, including low-level hardware data accessed through I<sup>2</sup>C communication, and periodically writes results

to disk, which introduces minor latency variation. The Python script, by comparison, records a smaller set of metrics, stores them temporarily in memory, and writes them to disk only after the monitoring session ends, resulting in more consistent intervals. Overall, both monitoring approaches achieve stable and reproducible timing behavior. This confirms that the dataset preserves accurate temporal alignment and consistent sampling characteristics across tools, ensuring the reliability of time-series measurements used in subsequent analyses.

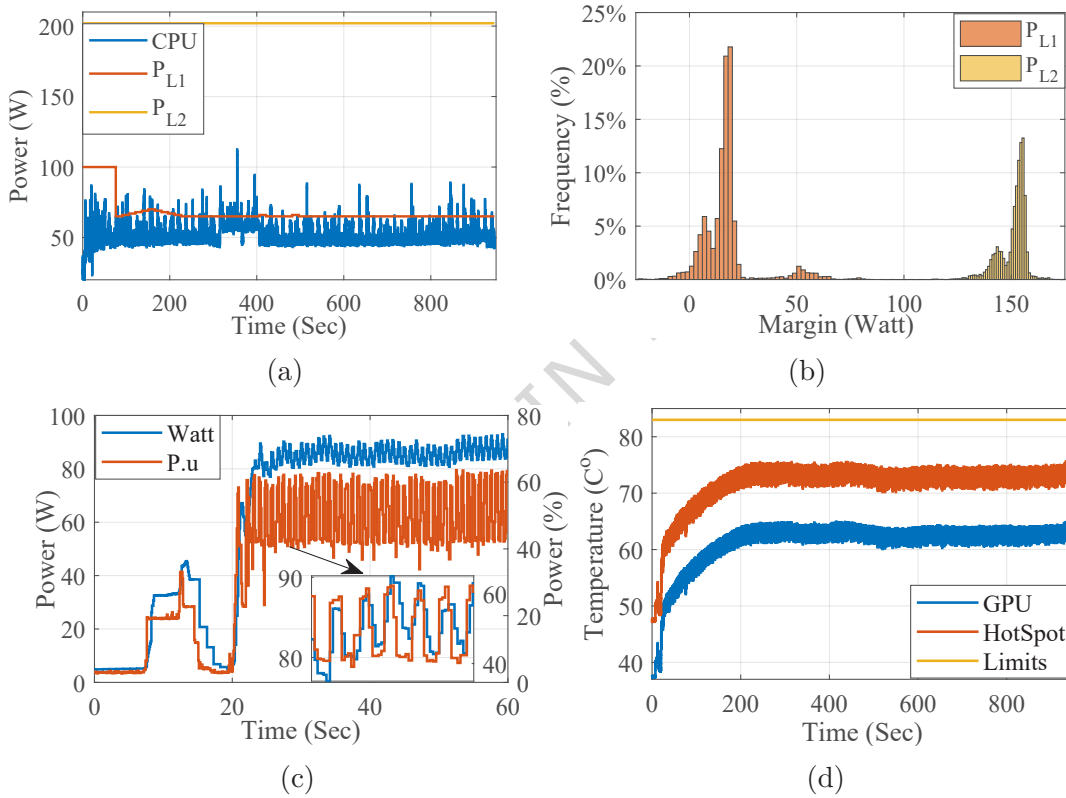


Figure 3: Operation Limits, (a) CPU power and dynamic power demand limits, (b) power margin distribution, (c) GPU power and percentage loading, and (d) GPU temperature with thermal limits analysis.

## Operational Validity

The observed power, temperature, utilization, and usage metrics remained within hardware-defined operational thresholds, demonstrating stable operation of both CPU and GPU throughout training.

Figure 3a shows the CPU power demand during AI training on a single machine plotted together with the long-term power limit ( $P_{L1} \approx 90W$ ) and the short-term boost power limit ( $P_{L2} \approx 200W$ ) defined by the processor firmware. The measured CPU power exhibited a median of approximately 50–55 W, remaining below  $P_{L1}$  for more than 95% of the time and below  $P_{L2}$  for the entire training duration. This confirms that no thermal or electrical throttling affected the recorded performance metrics, thereby supporting the integrity and reliability of the collected data. Figure 3b shows the statistical distribution of the CPU power headroom, defined as the instantaneous difference between the active CPU power and each power limit ( $P_{\text{limit}} - P_{\text{CPU}}$ ). The histogram illustrates that most samples have positive margins of approximately 30–40 W relative to  $P_{L1}$  and 130–150 W relative to  $P_{L2}$ , indicating that the CPU consistently operated well below both limits throughout training.

Similarly, Figure 3c demonstrates the GPU power demand and utilization percentage during training. As shown by the figure, GPU utilization averages approximately 40–60%, while the power demand hovers around 80–90 W. This behavior demonstrates consistent workload engagement and stable power delivery across the entire training cycle. Figure 3d displays the GPU thermal profile, including both average core temperature and hotspot temperature relative to the vendor-specified thermal limit ( $\approx 83^\circ\text{C}$ ). The average GPU temperature stabilized at 62–65  $^\circ\text{C}$ , while the hotspot temperature remained 72–75  $^\circ\text{C}$ , staying 7–10  $^\circ\text{C}$  below the thermal limit. The absence of thermal limit exceedance or throttling events confirms that the GPU operated within safe thermal margins throughout the experiment, which indicates that the recorded GPU performance metrics accurately represent undistorted workload characteristics. Together, these results confirm that all recorded CPU and GPU performance metrics were obtained within safe and stable operating conditions.

To ensure the operational validity of the CPU cores, both utilization and usage metrics are analyzed to verify that the cores operate within their expected ranges. CPU utilization is a performance-oriented metric that reflects the operating performance of a CPU relative to its base clock frequency. In contrast, CPU usage is a time-based metric that indicates

the proportion of time the CPU is actively executing tasks during a given reporting interval. CPU utilization can be interpreted as the number of clock cycles during which a CPU core operates in a computational state (commonly denoted as  $T_0$  and  $T_1$ ) relative to the processor's base clock frequency. Modern CPUs typically operate in two modes: base frequency mode and turbo frequency mode, where the processor temporarily increases its clock speed to handle computationally intensive workloads. As a result, CPU core utilization can exceed 100%, with a theoretical maximum equal to  $100 \times \frac{f^{GHz}_{Turbo}}{f^{GHz}_{Base}}$ . For the 12th Gen Intel(R) Core(TM) i7-12700 processor used in this study, the base and turbo clock frequencies are 2.1 GHz and 4.9 GHz, respectively, resulting in a maximum theoretical CPU utilization of 233.33%. In contrast, CPU usage is defined as the ratio of the time a CPU core spends in the computational state during the reporting period. Since this metric represents the fraction of time the core is active within the interval, its maximum value is bounded by 100%.

To provide a comprehensive assessment of the utilization of each CPU core and verify that the values remain within the expected operational range, Fig. 4a presents the utilization metrics for all CPU cores. The processor consists of 8 Performance cores (P-cores) and 4 Efficient cores (E-cores), and all observed utilization values remain below the theoretical limit of 233.33%. As expected, the P-cores exhibit higher utilization levels than the E-cores, as they are designed to handle computationally intensive tasks. In particular, P-Core 4 and P-Core 5 demonstrate the highest utilization values during workload execution. Similarly, Fig. 4b presents the usage metrics for all twelve CPU cores, with all values remaining below the theoretical upper bound of 100%. Consistent with expectations, the P-cores exhibit higher average usage compared to the E-cores, reflecting their role in executing compute-intensive operations. In addition, a strong correlation is observed between CPU core utilization and usage. Specifically, the Probability Distribution Function (PDF) of CPU usage across all cores follows a pattern similar to that of CPU utilization, where higher utilization generally corresponds to higher usage. This relationship is expected, as both metrics reflect the computational load imposed on the CPU cores.

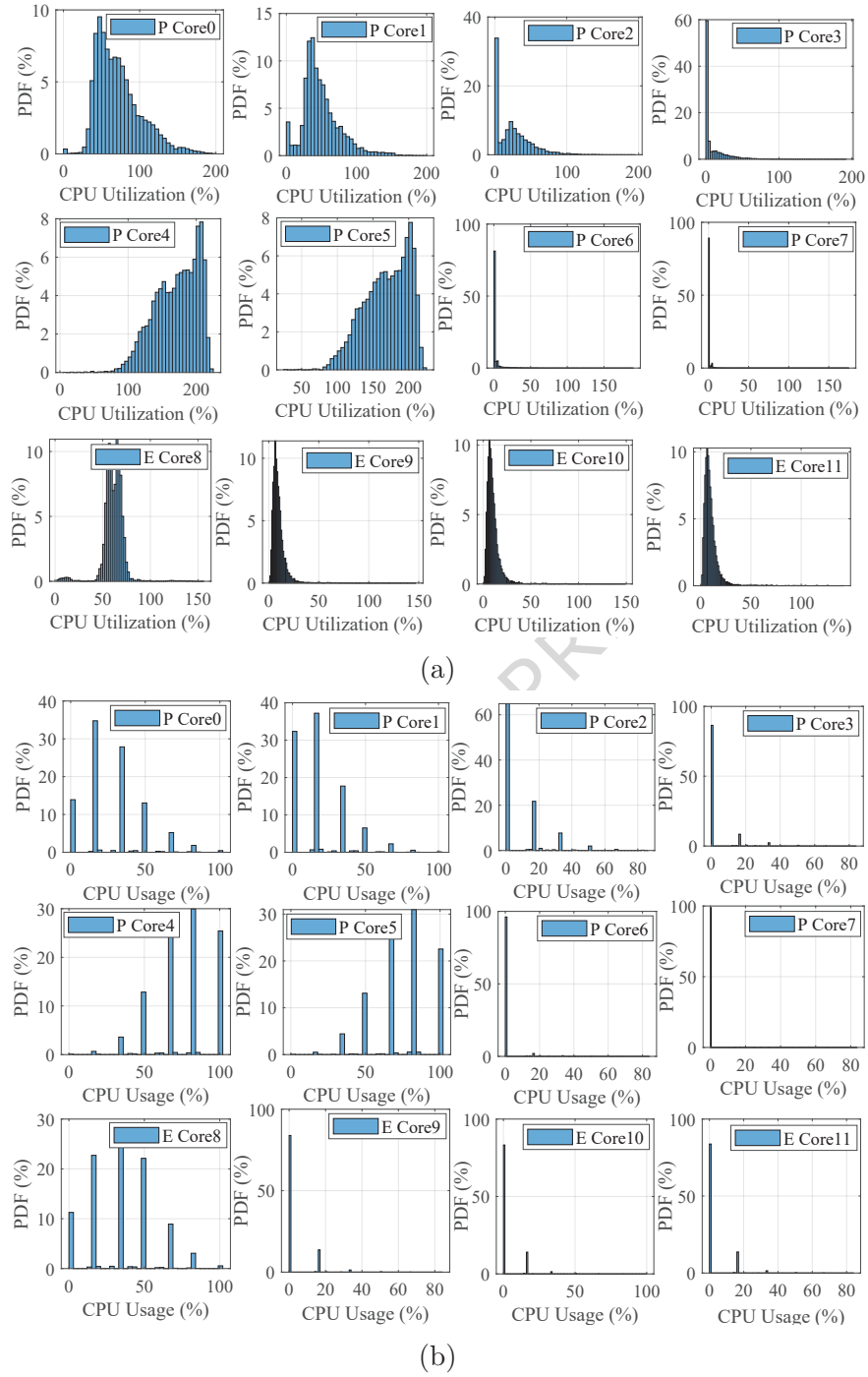


Figure 4: CPU core utilization and usage matrix (a) The CPU cores utilization lies within the expected operation range of 233.33%, (b) The CPU cores usage lies within the expected operation range of 100%

## Measurement Consistency

To verify the consistency and accuracy of the CPU power measurements, the total CPU package power was compared against the sum of the IA (integer and arithmetic) cores, GT

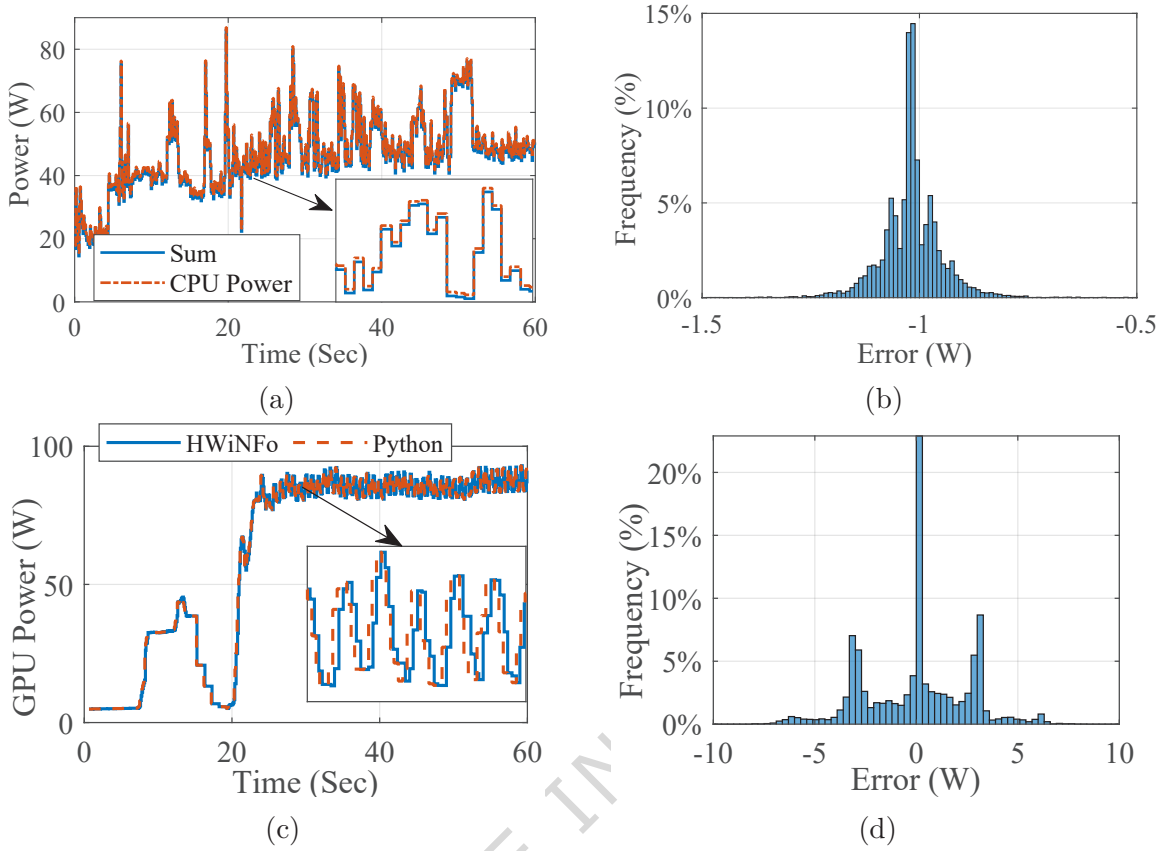


Figure 5: CPU and GPU measurement consistency: (a) CPU package power compared with the sum of core, GT core, system agent, and Rest-of-Chip power components, (b) corresponding sum error, (c) GPU power monitoring using HwiNFO and Python monitoring code, and (d) corresponding matching error.

(graphics and throughput) cores, system agent, and Rest-of-Chip power, each obtained from independent sensors sampled at the same frequency. As shown in Figure 5a, the aggregated sensor readings closely align with the total package power, confirming the internal power accounting accuracy of the monitoring framework. Figure 5b presents the corresponding error distribution. For 95% of the samples, the discrepancy ranges between -1.5 W and -0.5 W, which is within the expected measurement uncertainty. The slight negative bias likely reflects internal power losses and uninstrumented components within the CPU package. Repeated measurements on separate runs produced consistent results, demonstrating reproducibility of the power monitoring process across sessions and machines.

For GPU power validation, measurements obtained independently from HwiNFO and



the Python-based monitoring framework were compared to ensure consistency. Because the two tools operate at different reporting intervals (HwiNFO at 100 ms and the Python script at 20 ms), the following procedure was applied to enable synchronization and reporting rate alignment:

1. First, time synchronization is ensured so that both monitoring approaches reference the same system clock. When monitoring tasks are performed across different nodes or platforms, the Network Time Protocol (NTP) can be used to maintain clock synchronization across systems.
2. The recorded time vector is then converted from the HH:MM:SS.ms format into SS.ms using equation (1):

$$T_{SS.ms}^*(k) = T_{HH}(k) \times 3600 + T_{MM}(k) \times 60 + T_{SS.ms}(k) \quad (1)$$

Here,  $T_{\{\cdot\}}$  denotes the original time vector in the HH:MM:SS.ms format, while  $T_{SS.ms}^*$  represents the converted time vector expressed in seconds (SS.ms). This conversion places all timestamps within a continuous time-series window, facilitating consistent alignment and analysis of data collected from different sources.

3. All time vectors are subsequently aligned to a common reference point (zero point), defined as the minimum timestamp across the considered monitoring sessions. This reference is defined using equation (2):

$$T_{SS.ms}^{act}(i) = T_{SS.ms}^*(i) - T_{ref}, \quad \text{where } T_{ref} = \min(T_{SS.ms}^*(i) : i \in \mathcal{P}) \quad (2)$$

Here,  $T_{SS.ms}^{act}(i)$  denotes the aligned time vector for monitoring session  $i$ ,  $\mathcal{P}$  represents the set of monitoring sessions considered for comparison or aggregation across different platforms, and  $T_{ref}$  denotes the reference timestamp used to align all time series to a common starting point.

4. To address the mismatch in reporting rates, a Zero-Order Hold (ZOH) interpolation method is applied to estimate samples at a unified reporting frequency. In this approach, metrics from all monitoring sessions are resampled with respect to the highest reporting rate, ensuring consistent temporal alignment across datasets. ZOH interpolation preserves the last observed value until a new measurement becomes available, which is suitable for step-wise monitored signals such as power and utilization metrics.

It is worth noting that although these steps were applied here to compare the HWiNFO and Python monitoring results, the same procedure can also be used for data aggregation across different platforms or nodes, particularly in heterogeneous computing environments. Power metrics were specifically selected for cross-validation because both monitoring tools update power data at comparable frequencies, enabling precise temporal alignment.

Figure 5c presents the GPU power readings from both methods and their deviations, while Figure 5d shows the corresponding matching error. Across all workloads, 95 % of the samples exhibit consistency within 5 W, which falls within the expected combined uncertainty of the sensors and sampling synchronization. The minor residual discrepancies are primarily attributed to slight variations in the internal reporting intervals of the two monitoring systems. Together, Figures 5a–5d confirm the reliability and reproducibility of the GPU power measurements across independent hardware and monitoring tools.

Figures 6a and 6c present the validation of the total CPU utilization and usage, respectively. These metrics are defined as the average utilization and usage across all P-cores and E-cores and are compared with the values reported by HWiNFO. The results demonstrate strong agreement between the calculated values and the measurements reported by HWiNFO, confirming the accuracy and consistency of the monitoring results. As shown in Fig. 6b, the deviation between the two utilization measurements remains within  $\pm 0.1\%$ , which aligns with the expected mathematical relationship. Similarly, Fig. 6d shows that the deviation between the two usage measurements remains within  $\pm 0.25\%$ . These small deviations indicate a high level of consistency and validate the reliability of the CPU metric measurements.

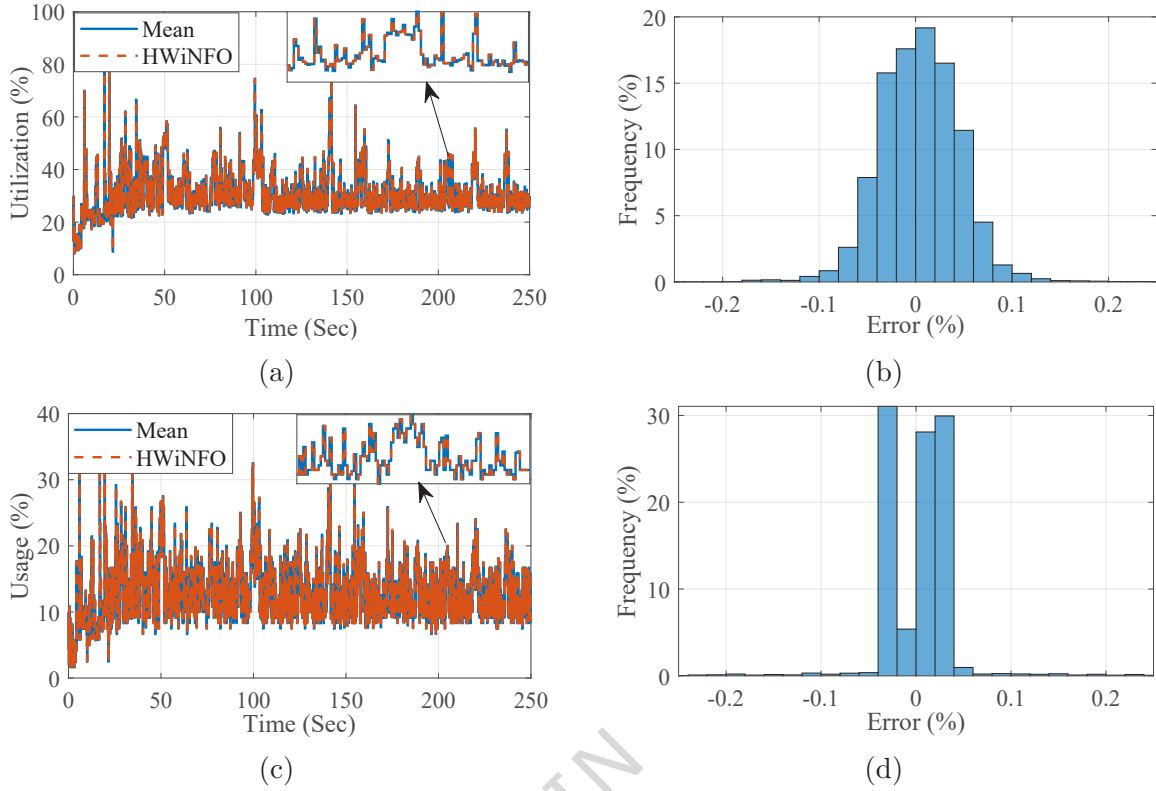


Figure 6: CPU Utilization and Usage consistency: (a) Total CPU utilization using mathematical calculation and HWiNFO reported values, (b) corresponding utilization error, (c) Total CPU usage using mathematical calculation and HWiNFO reported values, and (d) corresponding usage error.

## Physical Realism

This subsection validates the internal consistency and physical realism of the recorded CPU and GPU performance metrics by analyzing both temporal behavior and statistical correlations. Two validation processes were conducted: (1) short-term verification of the coherence among time-synchronized measurements and (2) assessment of the statistical relationships between key performance metrics to confirm physical plausibility.

### Physical Coherence of CPU Power Measurements

Figure 7a illustrates the temporal evolution of CPU power and utilization recorded during a feature regression training task. The CPU power varies between 20 W and 82 W, while utilization fluctuates between 18% and 82%. Distinct utilization peaks, such as those around

2 s and 4.2 s, correspond directly to increases in CPU power, confirming a strong temporal coupling between workload intensity and power demand. Figure 7b presents the variation of CPU power and temperature over time over a different instance. The temperature gradually rises from approximately 38 °C at the start of execution to around 50 °C as power increases, following the expected thermal response of the cooling system. A slight delay is observed between power spikes and temperature rise, consistent with thermal inertia in the heat dissipation mechanism. Figure 7c shows the behavior of CPU power and regulator current. As CPU power increases from 25 W to 95 W, the current drawn from the voltage regulator rises nearly linearly from approximately 40 A to 78 A. This relationship implies a nearly constant rail voltage in the range of 1.2–1.3 V, which agrees with expected CPU core voltage levels<sup>28</sup>. The consistency between power and current validates the electrical measurement integrity and supports the physical plausibility of the dataset.

To validate the statistical consistency of these relationships, the scatter plots in Figure 7d–7f show pairwise correlations among key variables. The correlation between CPU power and utilization ( $R = 0.91$ ) confirms that increased computational activity leads to higher energy draw. Core usage demonstrates a similar but slightly weaker correlation ( $R = 0.75$ ), reflecting parallel-thread scheduling effects. The relationship between power and temperature ( $R = 0.46$ ) exhibits a moderate positive trend, where temperature rise is moderated by thermal control mechanisms that maintain stability at higher power levels. The relationship between regulator current and power ( $R = 0.88$ ) demonstrates near-linearity, validating electrical coherence with  $P \approx V \times I$  under nearly constant supply voltage.

These results collectively demonstrate that the dataset accurately represents the physical interdependence among CPU power, utilization, temperature, and electrical current. The magnitudes and proportional variations observed across workloads conform to expected processor behavior during AI training, where increased computational load and current draw correspond to higher power consumption and thermal output. The strong linear correlations and stable temporal patterns indicate that the recorded measurements are physically coherent,

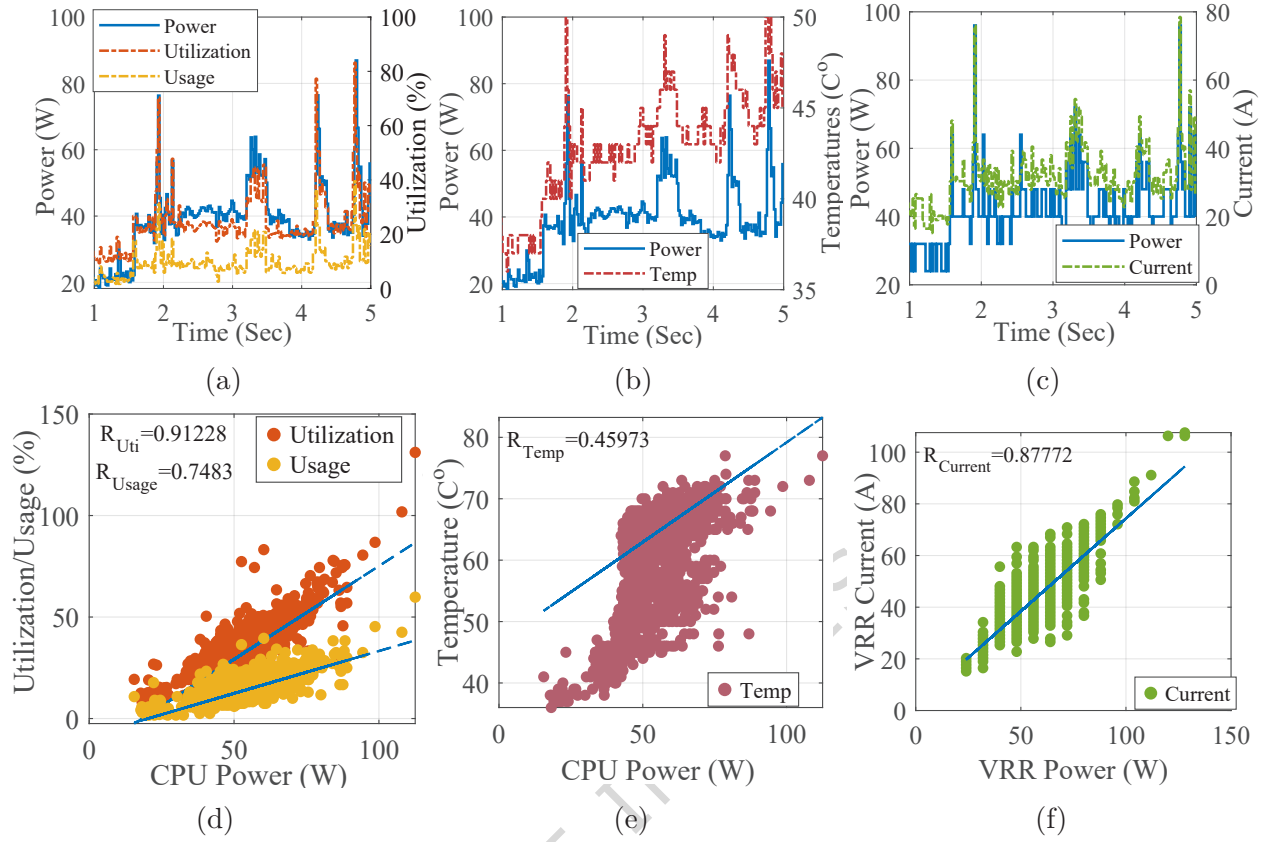


Figure 7: CPU performance metrics correlation: (a) power-utilization time series, (b) power-temperature time series, (c) power-current time series, (d) power-utilization correlation, (e) power-temperature correlation, and (f) power-current correlation.

internally consistent, and technically reliable.

### Physical Coherence of GPU Power Measurements

The dataset's internal consistency for GPU measurements was validated by examining both the time-series evolution of GPU parameters and their statistical correlations, as shown in Figure 8. These analyses confirm that the measured power, utilization, and temperature data accurately reflect the expected physical behavior of the GPU during AI workload execution.

Figure 8a illustrates the time-aligned trends between GPU power, core load, and memory load for a feature regression model training process. GPU power exhibits three major phases of activity between 0 and 30 seconds. During the initial phase (0–10 s), GPU power gradually increases from approximately 5 W to 43 W, corresponding to a moderate rise in core and

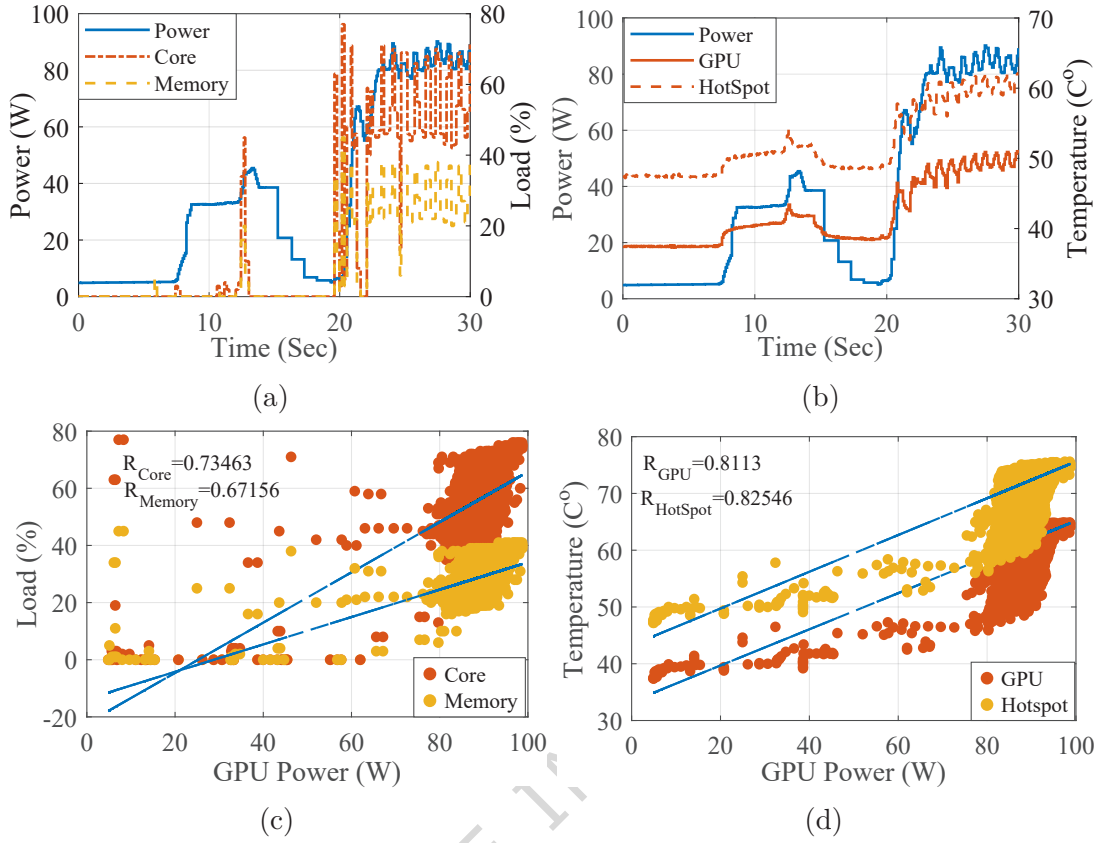


Figure 8: GPU performance metrics correlation, (a) GPU power, core, and memory load temporal evolution, (b) GPU power, temperature, and hotspot temperature temporal evolution, (c) GPU power and load correlation, and (d) GPU power and temperature correlation.

memory load from 0 % to 43 %. Between 10 and 20 seconds, both core and memory loads remain relatively stable while power dips momentarily, indicating reduced computational demand during. From 20 to 30 seconds, GPU power peaks at 80–90 W, while the core load rises sharply to 50–65 % and memory load hovers around 23–40 %. This synchronized escalation of power and load confirms that the GPU’s power behavior is dominated by active core operations, as expected under training workloads.

Consequently, GPU power demand and temperature also exhibit similar oscillatory patterns throughout training, as shown in Figure 8b, which depicts the relationship between GPU power and thermal metrics. As power rises from 8 W to 90 W, the GPU’s average temperature increases from approximately 38 °C to 50 °C, while the hotspot temperature (the highest sensor reading) climbs from 43 °C to 62 °C. The observed temperature rise of

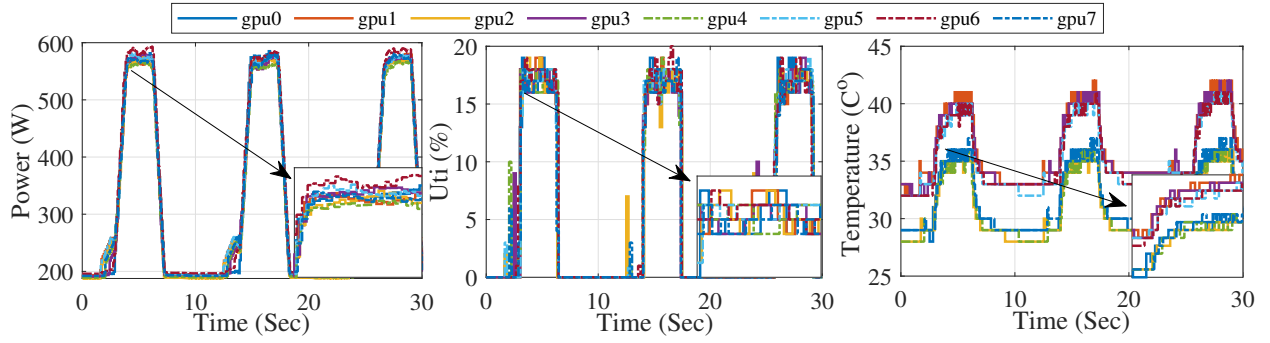


Figure 9: Multi-GPU timeseries pattern correlation

roughly 20 °C over a 80 W power increment aligns with typical GPU thermal characteristics under sustained load, suggesting proper thermal response and heat dissipation through active cooling. The temporal synchrony between power and temperature indicates that the dataset's timestamp alignment between electrical and thermal sensors is consistent.

To further validate these temporal observations, statistical correlation analyses were performed. Figure 8c shows scatter correlations between GPU power and load components. The Pearson correlation coefficient between GPU power and core load is 0.73, and between GPU power and memory load is 0.67, confirming strong positive associations. Figure 8d displays the relationships between GPU power and thermal parameters. The correlations are 0.81 for the average GPU temperature and 0.83 for the hotspot temperature, indicating that thermal and electrical metrics evolve coherently under increasing computational load. The observed magnitudes, proportional increases, and phase-aligned responses between power, utilization, and temperature confirm that the dataset captures physically plausible and internally consistent GPU behavior.

The AI training patterns are also evident in the multi-GPU node measurements. Figure 9 presents the key GPU performance metrics recorded during an image generation task using a diffusion model. The figure illustrates the oscillatory behavior and the consistency of these trends across all GPUs. These oscillations occur primarily during the training phases and reflect the cyclic nature of AI computation, alternating between feedforward and backpropagation passes and training epochs, which causes periodic variations in GPU utilization, power

| Metric             | GPU0 | GPU1 | GPU2 | GPU3 | GPU4 | GPU5 | GPU6 | GPU7 |
|--------------------|------|------|------|------|------|------|------|------|
| GPU Utilization    | 0.9  | 0.55 | 0.54 | 0.59 | 0.58 | 0.55 | 0.63 | 0.91 |
| Memory Utilization | 0.9  | 0.9  | 0.9  | 0.9  | 0.9  | 0.9  | 0.91 | 0.9  |
| Temperature        | 0.96 | 0.95 | 0.96 | 0.94 | 0.96 | 0.94 | 0.94 | 0.96 |

Table 5: GPUs power correlation with GPU utilization, memory utilization, and temperature

demand, and temperature<sup>5</sup>. The consistency of these oscillations across GPUs indicates that all devices are operating synchronously under the same workload distribution. Table 5 summarizes the correlation coefficients across the eight GPUs for different parameters. It should be noted that GPU0 has a different GPU power-utilization correlation factor because on NVIDIA multi-GPU systems, power usage depends on workload and configuration. GPUs with active clients or persistence mode remain powered, while idle GPUs may enter low-power or runtime-suspended states.<sup>29</sup>.

From a system dynamics perspective, the multi-GPU node exhibits stable and coordinated behavior throughout training. The observed power and temperature profiles show that all GPUs respond proportionally to workload changes, suggesting efficient synchronization and uniform task allocation by the deep learning framework. This consistency minimizes idle periods and load imbalance, ensuring optimal utilization of computational resources. Furthermore, the nearly identical thermal and power patterns across GPUs validate the reliability of the monitoring setup and highlight the dataset’s ability to reflect real-world multi-GPU operational characteristics. Such consistency is crucial for analyzing and characterizing AI workloads and their application in electric power system CIA.

Together, these analyses confirm both the physical and technical validity of the dataset across multiple execution environments, demonstrating its ability to accurately capture AI workload dynamics and inter-GPU interactions.

## Configuration Scaling

The scaling behavior of the dataset is evaluated to determine whether performance metrics remain consistent across different computational configurations. This analysis examines how



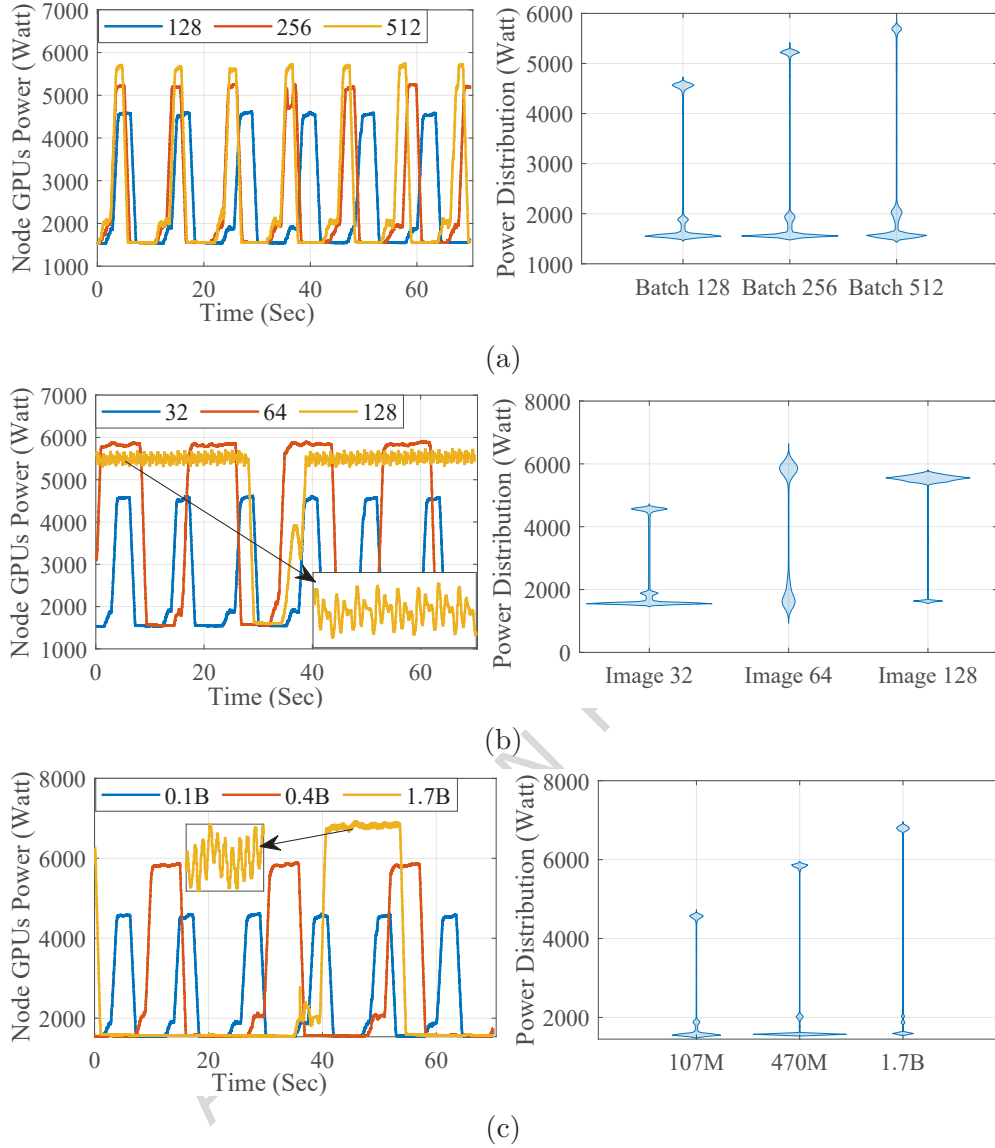


Figure 10: Correlation and power trends in a multi-GPU node under varying AI model architectures and hyperparameters during image generation training using a diffusion model on a B200 node: (a) batch size, (b) image size, and (c) model size.

these metrics vary with batch size, image resolution, and model complexity, verifying that the dataset reflects realistic and physically coherent scaling from single to multi-GPU operation.

Figure 10 presents the statistical analysis of the multi-GPU node, showing the total node power (sum of all individual GPU powers) and its distribution across different AI model architectures and hyperparameter configurations. The time-series and distribution plots reveal a distinct bimodal behavior, reflecting alternating low-power states during data

handling and high-power states during active computation.

Figure 10a illustrates the effect of batch size on node power. As the batch size increases from 128 to 512, the peak node power rises from approximately 4.5 kW to 6.6 kW, reflecting the higher computational load per iteration. Larger batches process more samples concurrently, which reduces the number of iterations per epoch and shortens the total epoch duration by approximately 30–35%. However, each iteration becomes more computationally intensive, resulting in higher instantaneous power consumption and longer high-power plateaus. The power distribution curves in the right-hand panel exhibit two dominant modes: a lower mode around 1.0–1.2 kW, corresponding to data preparation and synchronization phases, and an upper mode between 4.5 and 5.8 kW associated with GPU-intensive forward and backward propagation. As the batch size increases, the upper mode shifts upwards and narrows, indicating less dwell times in compute-bound states and more frequent transitions between synchronization and computation.

This observed behavior aligns with previously reported findings that larger batch sizes drive higher GPU utilization and increase average and peak power due to enhanced arithmetic intensity and memory throughput<sup>30</sup>. The dataset therefore captures realistic temporal and statistical characteristics of multi-GPU workloads, where larger batch sizes amplify instantaneous power demand and variability while maintaining coherent physical dynamics across all GPUs in the node.

Increasing input resolution leads to a near-quadratic expansion of the input layer, resulting in greater GPU memory consumption and computational load per iteration, as illustrated in the left panel of Figure 10b. As resolution rises from  $32 \times 32$  to  $128 \times 128$ , the node spends more time at high power and the plateaus lengthen. Peak power grows from about 4.4–4.6 kW at  $32 \times 32$  to a sustained plateau near 5.5–5.8 kW at  $128 \times 128$ , with the  $64 \times 64$  setting exhibiting the highest instantaneous peaks around 6.0–6.2 kW. Epoch and iteration times increase accordingly because each step processes larger tensors. The oscillatory traces reflect alternating forward and backward passes, and the longer transition intervals arise from

transferring and staging larger inputs. The distributions on the right panel of Figure 10b mirror these dynamics, where Image 32 shows a modest upper mode near 4.5 kW with a lower mode around 1.4–1.75 kW, while Image 64 broadens the upper mode and extends upper quantiles toward 6–7 kW. Meanwhile, Image 128 concentrates a narrow upper mode near 5.6 kW with a persistent low-power mode near 1.6–1.8 kW, indicating more sustained high-power dwell at a slightly lower peak ceiling. Overall, it is noted that higher resolutions increase work, driving greater GPU utilization and longer high-power duty cycles.

In addition to image size, the model size, represented by the number of learnable parameters, follows a similar pattern. Larger models require significantly more GPU memory and computational resources, as the increased number of parameters leads to more intensive processing during both feedforward and backpropagation phases. As illustrated in Figure 10c, increasing the model size from 107 million to 1.7 billion learnable parameters results in noticeably higher power demand during epoch training, along with longer epoch durations.

The observed trends from increasing image resolution and model size confirm that the dataset effectively captures both steady-state and transient power dynamics in multi-GPU environments, consistent with findings reported in the literature<sup>31</sup>, thereby providing a realistic representation of system-level power and performance characteristics under varying computational demands.

From the hardware type perspective, a B200 node offers greater GPU memory capacity and higher computational performance compared to an H100 node. When operating under identical hyperparameters, the B200 results in shorter epoch durations, as it can process larger workloads within the same time frame. However, this increased performance comes with a trade-off which is higher peak power demand. As shown in Figure 11a, greater processing capability leads to both higher amplitude and more frequent power fluctuations, producing stronger oscillations in the node’s overall power demand profile.

In contrast, modifications to parallelization settings have only a minor effect on the power characteristics. Parallelization primarily redistributes optimizer states, gradients, and

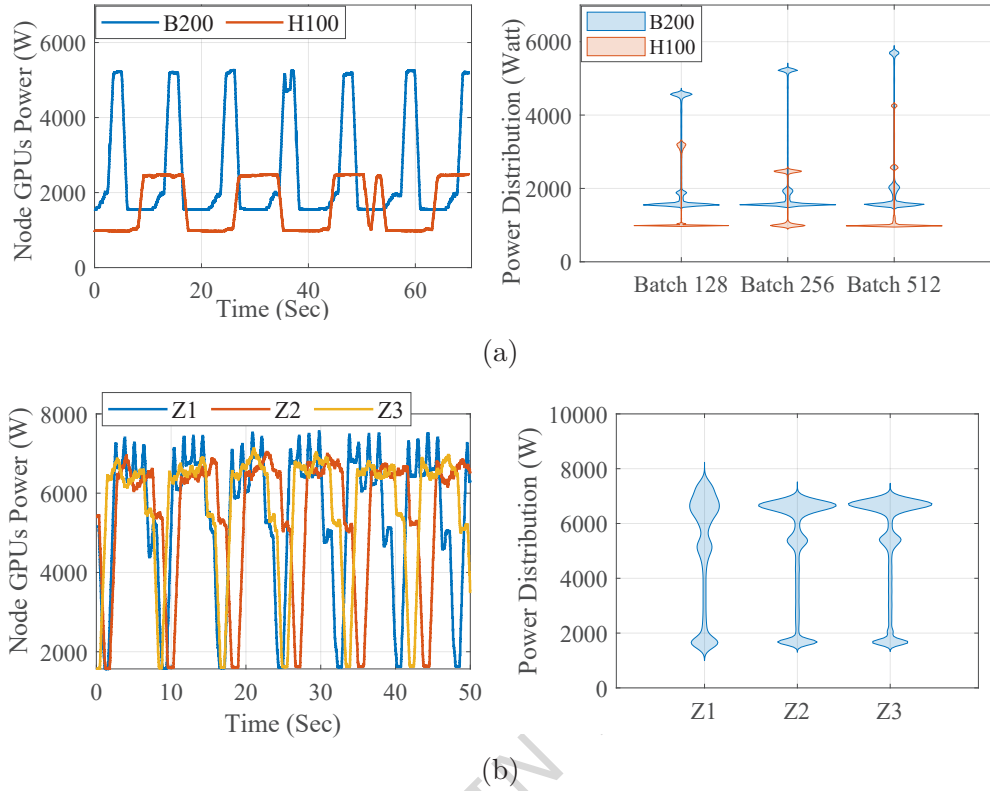


Figure 11: Power and correlation characteristics of a text generation workload in a multi-GPU node under varying node capabilities and parallelization configurations: (a) different hardware types and (b) parallelization settings using an LLM on a B200 node.

parameters across GPUs, introducing brief communication bursts between devices. Since it does not significantly alter per-GPU computational load during sustained processing phases, the average power and large-scale power dynamics remain nearly constant, as illustrated in Figure 11b. These observations reinforce the physical and technical validity of the dataset, confirming its ability to capture the distinct power and performance behaviors across different hardware configurations. The dataset thus provides a reliable foundation for AI workload characterization, energy modeling, and power quality analysis.

## Application-Oriented Validation

This section demonstrates the integrity and practical value of the proposed dataset by applying it to representative application scenarios. The objective is to validate the dataset through

real-world use cases that highlight both the technical challenges associated with large-scale AI datacenter integration and its potential for operational optimization. Specifically, the dataset is used to (1) assess the impact of large-scale AI datacenter workloads on power system operation through an Automatic Generation Control (AGC) case study, (2) support proactive control and operational planning through the development of a short-term forecasting model, and (3) assess and analyze the impact of different hyperparameter settings on energy and training efficiency. These applications illustrate how the dataset can assist in improving AI datacenter resource management as well as enable coordinated operation with power system operators to maintain system stability and ensure energy and training efficiency.

### **Power system Automatic Generation Control impact analysis**

An important application of the developed dataset is the assessment of the impact of large-scale AI data center integration on power system voltage and frequency stability. In this context, the AI workload profile can be incorporated into an AGC framework to evaluate how the integration of AI data center demand in one control area may influence the overall system frequency dynamics. To demonstrate this capability, the developed AI workload dataset is integrated into an AGC-based case study and its impact is compared with that of a wind farm operating at a similar MW scale. This comparison highlights the different dynamic characteristics and potential oscillatory effects that large AI data center loads may introduce to power system frequency stability.

Fig. 12 illustrates the two-area AGC system used in this case study, while Table 6 lists the adopted simulation parameters<sup>32</sup>. The two-area model is evaluated under three scenarios: (a) real-world wind farm power generation, (b) an AI datacenter operating under an image generation training workload, and (c) an AI datacenter operating under a LLM training workload. In these scenarios, the AI data center demand is connected to the load of Area 1, as illustrated in Fig. 13. Both the wind farm and the AI datacenter are assumed to have the same rated capacity of 1.7 p.u. For the image generation training workload, the B200 GPU

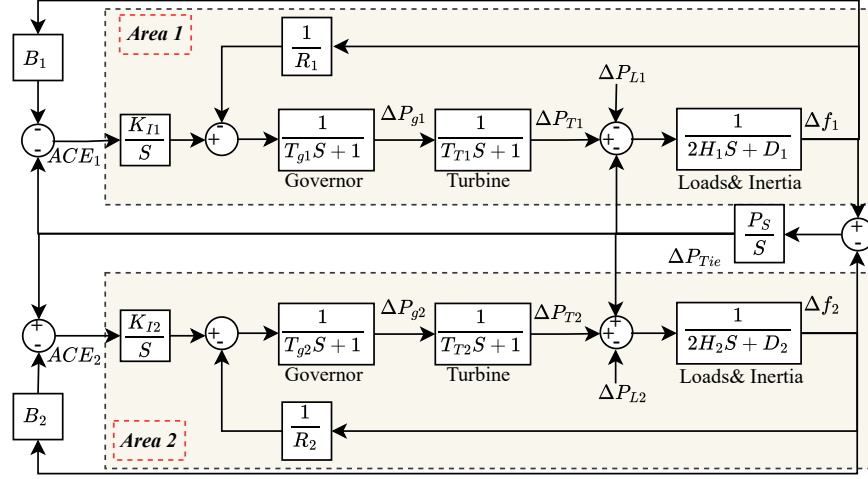


Figure 12: Block Diagram of a 2-area AGC system for AI datacenter connection impact assessment.

| Parameters      | Area 1                                | Area 2 |
|-----------------|---------------------------------------|--------|
| $D_i$ (pu/Hz)   | 0.6                                   | 0.3    |
| $H_i$ (sec)     | 5                                     | 4      |
| $R_i$ (Rad/pu)  | 0.05                                  | 0.0625 |
| $K_{I_i}$       | 0.3                                   | 0.3    |
| $T_{T_i}$ (sec) | 0.5                                   | 0.6    |
| $T_{g_i}$ (sec) | 0.2                                   | 0.3    |
| $B_i$ (HZ/pu)   | $\frac{1}{R_i} + D_i, i \in \{1, 2\}$ |        |
| $P_S$ (HZ/pu)   | 0.545                                 |        |

Table 6: AGC 2-Area system parameters

with a batch size of 512 is selected because it exhibits the highest rate of power variation, which may significantly influence AGC stability. Similarly, for the LLM training workload, the B200 GPU with a batch size of 32 using the LLaMA 8B model is chosen for the same reason, as it produces the most pronounced workload variability.

Fig. 14 presents the frequency deviation results of the AGC system under the three considered power profile scenarios. As shown, although wind farms are widely recognized as a major source of oscillatory behavior in power systems<sup>33</sup>, their impact on AGC frequency stability is significantly lower than that of the AI data center workload. This difference arises because, unlike GPU-based accelerators that can rapidly change their load from 20% to 100% within sub-second timescales, wind turbines contain rotating mechanical components

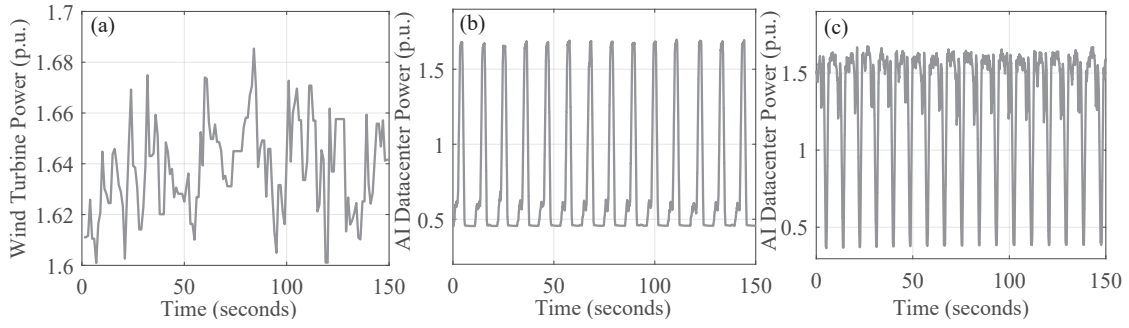


Figure 13: Area 1 power profiles under different operating scenarios: (a) wind farm generation, (b) AI image generation training workload, and (c) AI LLM training workload.

with inherent inertia that limit the rate of power variation. This effect is reflected in the frequency deviation ranges summarized in Table 7, where the maximum frequency deviation caused by the AI data center workload is approximately 16 times higher than that of the wind farm, despite both operating at the same rated power. Another notable observation is the oscillatory nature of the AI workload. The cyclic behavior of AI training processes introduces continuous fluctuations in the load profile, which leads to persistent oscillations in the AGC response and consequently larger overall frequency deviations.

Furthermore, due to the faster rate of power variation in AI workloads compared to wind generation, the Rate of Frequency Change (RoCoF), defined as  $\frac{\partial f}{\partial t}$ , is significantly higher under AI training conditions. When comparing the two AI workloads, the image generation training workload produces larger instantaneous frequency deviations because of its higher step changes in power demand. In contrast, the LLM training workload exhibits a greater number of training cycles, resulting in more frequent power fluctuations, which leads to larger long-term frequency variations and higher overall RoCoF. These results highlight the potential impact of directly connecting large-scale AI data centers to the power system without mitigation strategies. The findings emphasize the importance of incorporating local smoothing or buffering facilities within AI data centers to reduce rapid power fluctuations and improve grid stability.

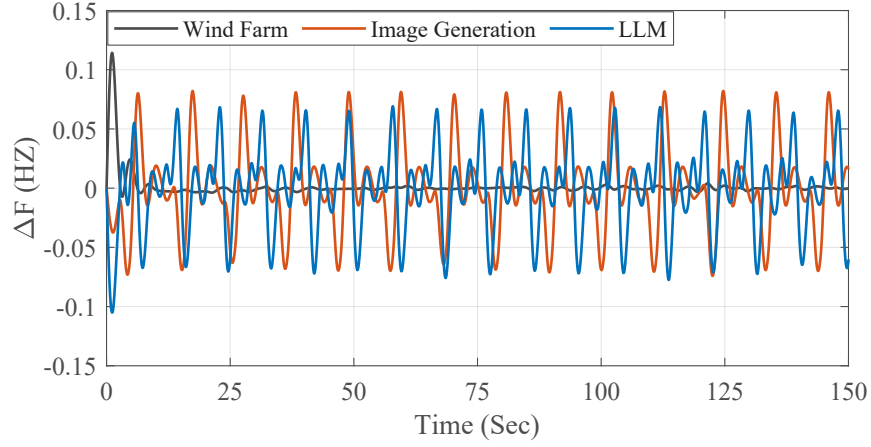


Figure 14: AGC frequency variation under wind farm generation, AI image generation training workload and, AI LLM training workload.

| Model            | $ \Delta f^{max} $ (Hz) | $\sum  \Delta f $ (Hz) | $\left  \frac{\partial f^{max}}{\partial t} \right $ (Hz/Sec) | $\sum \left  \frac{\partial f}{\partial t} \right $ (Hz/Sec) |
|------------------|-------------------------|------------------------|---------------------------------------------------------------|--------------------------------------------------------------|
| Wind farm        | 0.00497                 | 6.6482                 | 0.00654                                                       | 8.6576                                                       |
| Image Generation | 0.08347                 | 198.7364               | 0.12631                                                       | 279.3106                                                     |
| LLM              | 0.08331                 | 201.5682               | 0.14631                                                       | 331.7624                                                     |

Table 7: AGC frequency variation and RoCoF performance metric under wind farm generation, AI image generation training workload and, AI LLM training workload. The values calculated after 25 seconds to ensure initial condition dynamics are not accounted.

### AI datacenter short-term workload forecasting

Another important application of the developed dataset is the short-term forecasting of AI training workload power, which can support both AI data center operators and power system operators in optimal scheduling and impact mitigation. In this context, the AI data center workload profiles can be used to train time-series forecasting models, where a historical observation window ( $T^H$ ) is used to predict the expected AI workload over a future forecasting window ( $T^F$ ). To demonstrate this capability, the developed dataset is used to train a Long Short-Term Memory Deep Neural Network (LSTM-DNN) for short-term



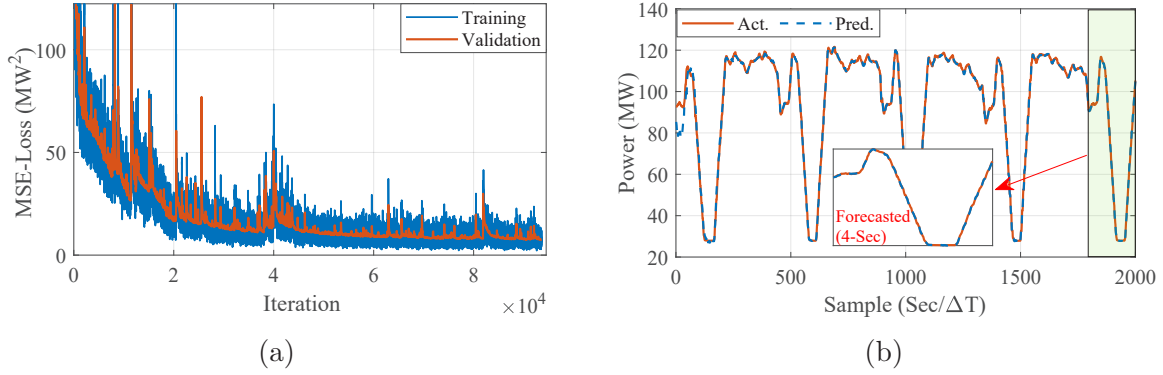


Figure 15: Short-term time-series forecasting results using LSTM-DNN model (a) training performance, (b) testing performance on new batch. The historical window is set for 40 seconds and forecasting window for 4 seconds.

time-series forecasting. The structure of the forecasting LSTM-DNN model is defined by (3).

$$\begin{aligned}
 P(t \in T^F) &= LSTM(P(t \in T^H)), \text{ where} \\
 LSTM^{L1}(X) &= \text{Norm}_{z\text{-score}}(\mathbf{x}) \\
 LSTM^{L2}(X) &= LSTM(|T^H|) \\
 LSTM^{L3}(X) &= \text{FC}(500) \\
 LSTM^{L4}(X) &= \text{ReLU}(X)
 \end{aligned} \tag{3}$$

Here,  $LSTM$  denotes the LSTM-DNN forecasting model, which takes the AI data center power measurements within the historical window ( $T^H$ ) as input and predicts the data center power over the forecasting window ( $T^F$ ). The term  $LSTM^{L1,2,3,4}$  represents the configuration of the model layers, including the layer number, type and its size.

Fig. 15 presents the performance of the short-term time-series forecasting model during both the training phase and the testing phase using unseen batch data. The training workload profile is scaled to 120 MW, and the model achieves a training error of approximately  $\pm 3$  MW after 9,000 iterations. In this configuration, the historical window is set to 40 seconds, with the objective of forecasting the workload over the next 4 seconds. Fig. 15b shows the predicted power profile over the forecasting window compared with the actual measured values. The

results demonstrate the capability of the developed dataset to support proactive scheduling and operational management of AI data centers, either through local resource coordination or in collaboration with power system operators to maintain voltage and frequency stability.

### AI datacenter workload energy and training efficiency evaluation

An additional contribution of the developed dataset is its capability to support energy consumption analysis, energy efficiency evaluation, and training efficiency assessment, enabling AI developers to optimize AI workload energy usage. In this context, two indices are introduced: (1) Learnable Epoch Energy (LEE), which quantifies the energy consumed per learnable parameter per epoch, and (2) Learnable Epoch Time (LET), which measures the training time per learnable parameter per epoch.

The LEE index is defined as

$$LEE = \frac{\sum_{t \in T} P^{Node}(t) \cdot \Delta t}{N^{Epoch} \cdot S^{Model}} \quad (4)$$

Here,  $\sum_{t \in T} P^{Node}(t) \cdot \Delta t$  represents the total energy consumed by the node during the training session,  $N^{Epoch}$  is the number of training epochs, and  $S^{Model}$  denotes the model size in terms of learnable parameters. A lower LEE value indicates higher energy efficiency, as it corresponds to less energy required to train a single parameter over one epoch. This formulation normalizes energy consumption with respect to model size, training duration, and hardware capability, allowing consistent comparison across different training configurations.

In contrast to LEE, which measures energy efficiency, the LET index represents training speed by quantifying the time required to update a single learnable parameter per epoch. The LET index is defined as

$$LET = \frac{\sum_{t \in T} \Delta t}{N^{Epoch} \cdot S^{Model}} \quad (5)$$

Here, LET is calculated as the total training time divided by the number of epochs and

| Index | H100       |        |        |            |        |        |            |        |        |
|-------|------------|--------|--------|------------|--------|--------|------------|--------|--------|
|       | Batch Size |        |        | Image Size |        |        | Model Size |        |        |
|       | 128        | 256    | 512    | 16         | 32     | 64     | 107M       | 470M   | 1.7B   |
| LEE   | 0.0681     | 0.0775 | 0.1035 | 0.0435     | 0.0681 | 0.1718 | 0.0681     | 0.0347 | 0.0262 |
| LET   | 0.1426     | 0.1682 | 0.2157 | 0.1121     | 0.1426 | 0.1912 | 0.1426     | 0.0598 | 0.0441 |

| Index | B200       |        |        |            |        |        |            |        |        |
|-------|------------|--------|--------|------------|--------|--------|------------|--------|--------|
|       | Batch Size |        |        | Image Size |        |        | Model Size |        |        |
|       | 128        | 256    | 512    | 32         | 64     | 128    | 107M       | 470M   | 1.7B   |
| LEE   | 0.0740     | 0.0709 | 0.0708 | 0.0740     | 0.1823 | 0.3095 | 0.0739     | 0.0365 | 0.0252 |
| LET   | 0.1065     | 0.1026 | 0.1020 | 0.1065     | 0.1682 | 0.2213 | 0.1065     | 0.0467 | 0.0311 |

Table 8: Hyperparameter impact on the energy efficiency and training speed defined by LEE and LET indices defined in  $\frac{\mu J}{Epoch \cdot Learnable}$  and  $\frac{\mu Sec}{Epoch \cdot Learnable}$ , respectively.

the number of learnable parameters. A lower LET value indicates a faster training process, corresponding to less time required per parameter update in each epoch.

Table 8 illustrates how both LEE and LET indices vary with changes in hyperparameters and machine type. For the batch size hyperparameter, the H100 exhibits an increasing trend in both energy consumption and training time (i.e., higher LEE and LET values) as the batch size increases. In contrast, the B200 shows an opposite trend, where increasing the batch size leads to reduced energy consumption and shorter training time. This difference is primarily attributed to GPU memory capacity and the ability to process the entire batch in a single pass. The B200, with its larger memory, can accommodate larger batch sizes without fragmentation. In contrast, the H100 may require batch splitting and gradient accumulation when memory limits are exceeded, which increases processing time, communication, and energy consumption. These results indicate that, from both energy and training efficiency perspectives, selecting a batch size within the GPU memory limit leads to improved LEE and LET indices.

From the image-size perspective, for both H100 and B200, increasing the image size (i.e., input layer dimension) at a fixed model size results in higher LEE and LET indices. This is because larger input dimensions require more computational operations at the input layer

and increase the overall data processing complexity. As a result, more energy is consumed during computation and longer training time is required. Therefore, reducing the input dimensionality, when feasible, such as through approaches like latent diffusion models, can improve both energy efficiency and training speed. On the other hand, increasing the model size results in improved energy and time efficiency per learnable parameter, as reflected by lower LEE and LET values. As the model size grows, the energy and time required to train a single parameter decrease. However, this does not imply a reduction in total energy consumption, as larger models contain more parameters and therefore require higher overall energy. In addition, model scalability is constrained by convergence behavior, since increasing model size does not necessarily guarantee improved performance.

Another observation emerges from comparing the H100 and B200 results under different hyperparameter settings. The B200 consistently achieves lower LET values, indicating faster training, but this advantage comes at the cost of higher LEE values and thus lower energy efficiency. This trade-off is primarily attributed to the higher power consumption and power density of the B200 GPUs. These results highlight the value of the developed dataset in guiding both power system and AI engineers toward selecting hyperparameter configurations that balance training speed and energy efficiency.

## Usage Notes

The dataset is designed to support multidisciplinary research on AI training workloads, datacenter energy behavior, and power system integration. Potential users include AI developers, datacenter engineers, and power system analysts. For AI developers and datacenter designers, the dataset enables characterization of workload-specific energy patterns, optimization of resource scheduling, and assessment of cooling and power provisioning strategies. For power system researchers, it provides realistic, high-resolution input for CIA studies of large-scale AI datacenter integration into electrical grids.

Users should note that ambient temperature and datacenter cooling configurations are not included and may influence absolute thermal readings in other settings.

## Data Availability

The dataset generated and analyzed in this study is publicly available through an open-access repository shared via figshare "<https://doi.org/10.6084/m9.figshare.31654879>". It includes detailed measurements of power demand, CPU and GPU utilization, per-GPU power, memory usage, and temperature across AI training workloads on single and multiple GPU nodes. All data supporting the findings of this study are available within the repository and accompanying metadata files<sup>24</sup>.

## Code Availability

All codes used for 1) single-machine and node-level data generation and training workload execution, 2) Python-based monitoring and recording, and 3) data analysis and visualization are available in the GitHub repository<sup>25</sup>.

## Author contributions statement

A.A.E.E. designed and led the study, conducted the experiments, processed the data, and prepared the main manuscript text. A.A.A. contributed to data acquisition, validation, and analysis of datacenter power and system performance. H.E.Z.F. supervised the project, contributed to methodology design, interpretation of results, and manuscript revision. All authors reviewed and approved the final manuscript.

## Competing Interests

The authors declare no competing interests.

## Funding

This work has been funded by the Natural Sciences and Engineering Research Council of Canada (NSERC).

## References

- [1] Chen, Sophia, How much energy will AI really consume? The good, the bad and the unknown. Nature 639.8053, 22-24, (2025).
- [2] Rasch MJ, Mackin C, Le Gallo M, Chen A, Fasoli A, Odermatt F, Li N, Nandakumar SR, Narayanan P, Tsai H, Burr GW, Hardware-aware training for large-scale and diverse deep learning inference workloads using in-memory computing-based accelerators. Nature communications, 14(1), 5282, (2023).
- [3] Alarcon, Nefi, OpenAI presents GPT-3, a 175 billion parameters language model. Nvidia <https://developer.nvidia.com/blog/openai-presents-gpt-3-a-175-billion-parameters-language-model/> (2020).

- [4] Liu A, Feng B, Xue B, Wang B, Wu B, Lu C, Zhao C, Deng C, Zhang C, Ruan C, Dai D. Deepseek-v3 technical report. Preprint at <https://arxiv.org/pdf/2412.19437> (2024).
- [5] North American Electric Reliability Corporation (NERC), Characteristics and Risks of Emerging Large Loads [https://www.nerc.com/comm/RSTCReviewItems/3\\_Doc\\_White%20Paper%20Characteristics%20and%20Risks%20of%20Emerging%20Large%20Loads.pdf](https://www.nerc.com/comm/RSTCReviewItems/3_Doc_White%20Paper%20Characteristics%20and%20Risks%20of%20Emerging%20Large%20Loads.pdf) (2025).
- [6] Tazi, Nouamane, Ferdinand Mom, Haojun Zhao, Phuc Nguyen, Mohamed Mekouri, Leandro Werra, and Thomas Wolf. The Ultra-Scale Playbook: Training LLMs on GPU Clusters (2025).
- [7] Wang, Qiang, Fuyu Zhang, and Rongrong Li, Artificial intelligence and sustainable development during urbanization: Perspectives on AI R&D innovation, AI infrastructure, and AI market advantage. Sustainable Development 33.1, 1136-1156, (2025).
- [8] Google Cluster Data <https://github.com/google/cluster-data> (2019).
- [9] Azure Public Dataset <https://github.com/Azure/AzurePublicDataset> (2019).
- [10] Alibaba Cluster Trace Program <https://github.com/alibaba/clusterdata> (2023).
- [11] Antici, Francesco, Mohsen Seyedkazemi Ardebili, Andrea Bartolini, and Zeynep Kiziltan, PM100: A job power consumption dataset of a large-scale production HPC system. In Proceedings of the SC'23 Workshops of The International Conference on High Performance Computing, Network, Storage, and Analysis, pp. 1812-1819 (2023).
- [12] Antici, Francesco, Andrea Bartolini, Jens Domke, Zeynep Kiziltan, and Keiji Yamamoto, F-DATA: A Fugaku Workload Dataset for Job-centric Predictive Modelling in HPC Systems. Scientific Data 12, no. 1, 1321 (2025).
- [13] Profiling Data in DeepSeek <https://github.com/deepseek-ai/profile-data> (2024).

- [14] Tu, Xiaolong, Anik Mallik, Haoxin Wang, and Jiang Xie. Deepen2023: Energy datasets for edge artificial intelligence. preprint <https://arxiv.org/pdf/2312.00103> (2023).
- [15] Department of Energy, BUTTER-E - Energy Consumption Data for the BUTTER Empirical Deep Learning Dataset <https://catalog.data.gov/dataset/butter-e-energy-consumption-data-for-the-butter-empirical-deep-learning-dataset> (2024).
- [16] Yousefzadeh-Asl-Miandoab, Ehsan, Ties Robroek, and Pinar Tozun, Profiling and monitoring deep learning training tasks. In Proceedings of the 3rd Workshop on Machine Learning and Systems, pp. 18-25 (2023).
- [17] Lambda AI, On Demand GPU Instances <https://docs.lambda.ai/public-cloud/on-demand/>.
- [18] Deep Learning Neural Network for Classification <https://www.mathworks.com/help/deeplearning/ug/create-simple-deep-learning-network-for-classification.html> (2025).
- [19] Word-by-Word Text Generation Using Deep Learning <https://www.mathworks.com/help/deeplearning/ug/word-by-word-text-generation-using-deep-learning.html> (2025).
- [20] Image Captioning Using Attention <https://www.mathworks.com/help/deeplearning/ug/image-captioning-using-attention.html> (2025).
- [21] Train a diffusion model with PyTorch Lightning <https://lightning.ai/lightning-ai/studios/train-a-diffusion-model-with-pytorch-lightning?section=feature> (2024).
- [22] LLaMA Factory, Unified Efficient Fine-Tuning of 100+ LLMs & VLMs <https://github.com/hiyouga/LLaMA-Factory> (2024).



- [23] DeepSpeed, Zero Redundancy Optimizer <https://www.deepspeed.ai/tutorials/zero/> (2025).
- [24] A.A. Elsayed, A.A. Al-Obaidi, Hany Farag, High-resolution-AI-Data-Center-Training-Workloads-Dataset, <https://doi.org/10.6084/m9.figshare.31654879> (2026).
- [25] A.A. Elsayed, A.A. Al-Obaidi, Hany Farag, High-resolution-AI-Data-Center-Training-Workloads-Dataset Codes, <https://github.com/Ahmed-Elsayed95/High-resolution-AI-Data-Center-Training-Workloads-Dataset/tree/main> (2025).
- [26] HWiNFO, Comprehensive Hardware Analysis, Monitoring and Reporting for Windows and DOS <https://www.hwinfo.com/about-software/>.
- [27] Lambda AI, GPU Instances <https://lambda.ai/>.
- [28] Intel Corporation. *12th Generation Intel® Core™ Processor Families Datasheet, Volume 1*. Document Number: 655258-011, January 2024. Available at: <https://www.intel.com/content/www/us/en/content-details/655258/intel-core-processor-datasheet-vol-1.html>
- [29] NVIDIA, “Chapter 22. PCI-Express Runtime D3 (RTD3) Power Management,” *NVIDIA Driver Documentation*, version 550.67, 2025. [Online]. Available: [https://download.nvidia.com/XFree86/Linux-x86\\_64/550.67/README/dynamicpowermanagement.html](https://download.nvidia.com/XFree86/Linux-x86_64/550.67/README/dynamicpowermanagement.html). [Accessed: Oct. 2025].
- [30] S. M. Nabavinejad, S. Reda, and M. Ebrahimi, “BatchSizer: Power-Performance Trade-off for DNN Inference,” in *Proc. 26th Asia and South Pacific Design Automation Conf. (ASPDAC '21)*, Tokyo, Japan, 2021, pp. 819–824. doi: 10.1145/3394885.3431535.
- [31] E. Strubell, A. Ganesh, and A. McCallum, “Energy and Policy Considerations for Modern Deep Learning Research,” in *Proc. AAAI Conf. Artif. Intell.*, vol. 34, no. 9, pp. 13693–13696, Apr. 2020.

- [32] Khalaf M, Youssef A, El-Saadany E. Joint detection and mitigation of false data injection attacks in AGC systems. *IEEE Transactions on Smart Grid*. 2018 Sep 26;10(5):4985-95.
- [33] El-Hamalawy AF, Farag HE, Ahmed E, Sohm D, El-samahy I. A Novel Framework for Centralized Remote Power Smoothing as a Prospective Ancillary Service. *IEEE Transactions on Sustainable Energy*. 2025 Dec 8.

ARTICLE IN PRESS